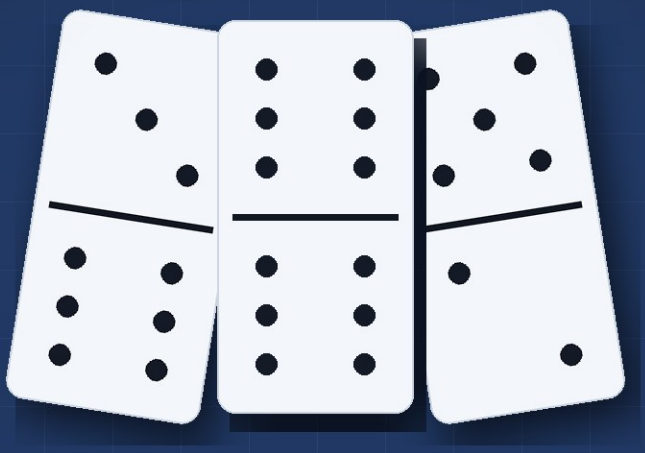


BUKU PENJELASAN & PANDUAN LENGKAP

Domino Analyzer Pro

Panduan pemakaian, penjelasan mesin analisis Monte-Carlo, rujukan API, dan strategi bermain domino 28 kartu.

by Dody Maulana Ramadhan

Versi aplikasi v3.3 · engine 3.0.0

Build dokumen 2026-10-15

10 Bagian · 9 diagram · 75 halaman

Daftar Isi

BAGIAN I — Pengenalan 5

1.1 Apa itu Domino Analyzer Pro 5

1.2 Masalah yang dipecahkan 6

1.3 Fitur utama dalam satu halaman 6

1.4 Cara memakai dalam 30 detik 7

1.5 Cara membaca buku ini 7

1.6 Pernyataan jujur tentang batasan 8

BAGIAN II — Dasar Permainan Domino 9

2.1 Kartu: 28 klik dari 0 sampai 6 9

2.2 Meja: rantai dua ujung 10

2.3 PASS: ketika tidak bisa jalan 10

2.4 Cangkul: menyambar dari sisa 10

2.5 Adu: ketika meja macet 11

2.6 Jumlah pemain dan arah giliran 11

2.7 Istilah ringkas 12

BAGIAN III — Memasang & Menjalankan 13

3.1 Kebutuhan minimum 13

3.2 Menjalankan dalam satu perintah 13

3.3 Membangun mesin secara manual 14

3.4 Menjalankan server secara terpisah 15

3.5 Konfigurasi lewat variabel lingkungan 15

3.6 Membuka akses dari internet (opsional) 16

3.7 Memasang sebagai aplikasi (PWA) 16

3.8 Struktur proyek 17

3.9 Menguji pemasangan Anda 18

BAGIAN IV — Panduan Antarmuka 19

4.1 Tata letak layar 19

4.2 Bilah judul 19

4.3 Panel Setup: memberi tahu aturan meja 20

4.4 Bilah aksi 22

4.5 Papan: meja domino 22

4.6 Tangan Anda 23

4.7 Memasukkan kartu: modal 28 kartu dan masukan cepat 24

4.8 Mencatat lawan: panel Lawan 24

4.9 Peta angka 25
4.10 Kartu sisa: tiga perilaku berbeda 26
4.11 Panel hasil analisis 26
4.12 Banner kualitas kartu 27
4.13 Banner menang 27
4.14 Mode hidup, undo, dan pengelolaan sesi 27
4.15 Pesan kesalahan dan artinya 28
BAGIAN V — Membaca & Memercayai Hasil 29
5.1 Urutan kartu ditentukan mesin, bukan tampilan 29
5.2 Contoh nyata dengan angka asli 30
5.3 Kunjungi satu kartu secara lengkap 31
5.4 Interval keyakinan: alat ukur keraguan 32
5.5 Peringkat mana yang boleh diabaikan 33
5.6 Profil kekayaan angka lawan 33
5.7 Kartu sisa yang paling mungkin mati 34
5.8 Mode kartu pertama: membaca keamanan putaran kedua 34
5.9 Apa yang tidak bisa diberitahu oleh angka ini 35
BAGIAN VI — Arsitektur & Isi Mesin 36
6.1 Tiga lapisan 36
6.2 Alur satu permintaan 37
6.3 Mesin: satu berkas C++20 tanpa dependensi 38
6.4 Mesin: satu langkah dianalisis bagaimana 39
6.5 Mesin: satu gim simulasi 41
6.6 Mesin: dari hasil ke rekomendasi 41
6.7 Server: tipis, tetapi tidak bodoh 42
6.8 Antarmuka: satu berkas, satu sumber keadaan 44
6.9 Tata letak papan: mengapa ada "pengecilan otomatis" 44
6.10 Siluet rekomendasi (ghost) 45
6.11 Mode hidup, antrean, dan pembatalan 45
6.12 Service worker: dua strategi berbeda 46
6.13 Riwayat versi 46
BAGIAN VII — Pengujian & Kualitas 47
7.1 Sebelas pemeriksaan mandiri mesin 47
7.2 Dua puluh sembilan uji server 48
7.3 Paralelisasi: output harus tidak berubah (4 tes) 49
7.4 Verifikasi identik terhadap build sebelumnya 50
7.5 Mengukur kecepatan 51
7.6 Uji antarmuka tanpa kepala (6 tes) 52
7.7 Integrasi berkelanjutan (CI) 53

7.8 Bagaimana membuktikan hasil analisis bisa dipercaya 53
7.9 Sebuah cacat nyata: rumus interval keyakinan 54
7.10 Yang belum tercakup oleh pengujian otomatis 54
BAGIAN VIII — Panduan Strategi 56
8.1 Empat prinsip dasar 56
8.2 Mendominasi ujung 56
8.3 Jerat: membuat lawan membuka jalan Anda 57
8.4 Kunci: menutup jalan lawan 57
8.5 Adu: bermain dengan sisa mata 57
8.6 Cangkul: menyambar dengan sadar biaya 58
8.7 Menyesuaikan diri dengan gaya lawan 58
8.8 Rutinitas satu langkah 58
8.9 Kesalahan umum dan cara menghindarinya 59
8.10 Membaca situasi dengan cepat 59
BAGIAN IX — Rujukan API 60
9.1 Aturan dasar 60
9.2 Daftar endpoint 60
9.3 GET /api/health 61
9.4 POST /api/meta 61
9.5 POST /api/analyze — permintaan 62
9.6 POST /api/analyze — balasan 64
9.7 POST /api/selftest 66
9.8 Contoh lengkap dengan curl 66
9.9 Kode kesalahan 67
9.10 Memanggil mesin langsung 67
BAGIAN X — Pemecahan Masalah & Lampiran 69
10.1 Pemecahan masalah 69
10.2 Pertanyaan yang sering diajukan 70
10.3 Glosarium 71
10.4 Lampiran A — seluruh 28 kartu 72
10.5 Lampiran B — pembagian kartu dan sisa 72
10.6 Lampiran C — rumus dan bobot penting 73
10.7 Lampiran D — catatan versi dan perbaikan 73
10.8 Lampiran E — membangun ulang dokumen ini 74
10.9 Lisensi dan kontribusi 74
10.10 Tentang penulis 75
10.11 Penutup 75

BAGIAN I

Pengenalan

I

Selamat datang. Buku ini menjelaskan **Domino Analyzer Pro** *dari dua sisi sekaligus: sebagai panduan pemakaian sehari-hari, dan sebagai penjelasan teknis isi mesinnya. Tujuannya satu: setelah membaca, Anda bisa memakai aplikasi ini dengan percaya diri, tahu* **mengapa** *angka di layar muncul, dan tahu* **apa yang boleh dan tidak boleh** *dipercaya dari angka tersebut.*

Buku ini ditulis untuk pembaca yang tidak harus punya latar belakang pemrograman. Istilah teknis selalu dijelaskan saat pertama muncul, dan semua istilah penting dikumpulkan kembali di glosarium pada bagian lampiran.

1.1 Apa itu Domino Analyzer Pro

Domino Analyzer Pro adalah alat bantu keputusan untuk permainan **domino 28 kartu** (kartu dengan sisi angka 0 sampai 6, biasa disebut "klik" atau "gaple" di Indonesia). Anda memasukkan kartu yang ada di tangan Anda, menandai kartu yang sudah terlihat di meja, mencatat saat lawan tidak bisa jalan (PASS), lalu aplikasi menghitung kemungkinan menang untuk setiap langkah yang mungkin Anda ambil.

Perhitungannya bukan tebakan dan bukan kumpulan tips. Aplikasi ini benar-benar memainkan ribuan gim domino di dalam komputer: setiap gim dibagikan ulang secara acak dengan batasan yang Anda berikan, dimainkan sampai selesai oleh tiga gaya lawan yang berbeda, dan hasil akhirnya dihitung. Setelah semua percobaan selesai, aplikasi menyusun rekomendasi urutan langkah terbaik untuk Anda.

Komponen utamanya tiga:

Lapisan	Berkas	Peran
Antarmuka web	web/index.html, web/app.js, web/styles.css, web/sw.js	papan, tangan, pencatatan lawan, tampilan hasil
Server API	server/domino_server.py	menerima permintaan, menormalkan dan memvalidasi, memanggil mesin
Mesin analisis	engine/domino_engine.cpp → bin/domino_engine	simulasi Monte-Carlo, penilaian langkah, peringkat

Semuanya berjalan lokal di komputer Anda. Tidak ada data permainan yang dikirim ke layanan pihak ketiga; satu-satunya lalu lintas jaringan adalah antara peramban Anda dan server lokal di `127.0.0.1:2024`.

1.2 Masalah yang dipecahkan

Domino terlihat sederhana: tujuh kartu di tangan, dua ujung meja, siapa cepat habis dia menang. Kenyataannya ada tiga sumber kebingungan yang membuat pemain kuat sekalipun sering salah langkah.

Pertama, informasi tersembunyi terlalu banyak. Ada 28 kartu, dan Anda hanya tahu tujuh di antaranya. Setiap keputusan sebenarnya adalah perjudian terukur: peluang kartu tertentu ada di tangan lawan. Manusia buruk dalam menimbang banyak peluang sekaligus, terutama dengan batas waktu beberapa detik.

Kedua, informasi tersedia tidak dicatat. Setiap kali lawan menolak jalan (PASS), mereka membocorkan sesuatu yang sangat berharga: angka di kedua ujung meja saat itu **tidak ada** di tangan mereka. Satu PASS bisa menghapus belasan kemungkinan. Hampir semua pemain membuang informasi ini karena tidak mungkin menghafalnya.

Ketiga, hasil tipis tidak bisa dibedakan dari keberuntungan. Dua langkah yang hampir sama bagusnya akan menghasilkan win rate yang mirip. Tanpa alat ukur ketidakpastian, pemain cenderung menganggap selisih satu-dua persen sebagai alasan kuat, padahal itu hanya derau statistik.

Domino Analyzer Pro menyerang ketiga hal itu: mencatat bukti (PASS dan atribusi kartu), menghitung peluang dengan simulasi berskala besar, dan menampilkan ketidakpastian secara eksplisit dalam bentuk interval keyakinan.

1.3 Fitur utama dalam satu halaman

Mesin analisis. Simulasi Monte-Carlo untuk setiap kandidat langkah; tiga profil lawan AI (Blocker, Hoarder, Dumper) yang dirotasi sehingga rekomendasi tidak terjebak pada satu gaya main; mode **teli ti** (adversarial) di mana lawan yang bermain tepat setelah Anda membalas dengan permainan ketat tanpa derau; simulasi **cangkul**; konvensi meja yang bisa diatur (aturan adu dan aturan seri); interval keyakinan Wilson 95%; peringkat tunggal `rankScore` yang dihitung di mesin; serta hasil yang **deter ministik** — seed yang sama menghasilkan angka yang sama persis.

Pelacakan lawan. PASS lawan menghapus angka dari kemungkinan kartu mereka; atribusi kartu memastikan satu kartu milik satu lawan tertentu dan menghapusnya dari lawan lain; profil kekayaan angka per lawan menaksir angka mana yang kemungkinan besar mereka kuasai; dan urutan giliran ikut diperhitungkan dalam simulasi.

Antarmuka. Papan bergaya ular yang mengecil otomatis agar semuanya terlihat tanpa menggulir; siluet rekomendasi yang menempel di kepala atau kaki kartu pertama; mode LIVE yang menganalisis ulang setiap kali informasi berubah; timeline undo yang konsisten; simpan otomatis ke penyimpanan peramban; ekspor dan impor sesi; banner kualitas kartu; serta kemampuan dipasang sebagai aplikasi (PWA) yang tetap membuka antarmuka saat offline.

1.4 Cara memakai dalam 30 detik

Untuk pembaca yang ingin langsung mencoba:

```
python start.py
```

Perintah di atas membangun mesin C++ bila perlu, menjalankan pemeriksaan mandiri mesin, lalu menyalakan server di port 2024. Buka `http://127.0.0.1:2024` di peramban.

Lalu di layar:

1. Pilih jumlah pemain, jumlah kartu per pemain, dan aturan meja di panel **Setup**.
2. Ketuk + **Pilih Kartu Saya**, lalu isi tujuh kartu tangan Anda (bisa lewat ketikan cepat: dua angka lalu Enter).
1. Ketuk kartu di tangan untuk memainkannya. Bila kartu bisa masuk ke dua ujung, muncul pilihan ujung kiri atau kanan.
1. Saat giliran lawan: kalau dia bermain, ketuk kartunya di panel **Kartu Sisa** lalu pilih lawan yang memainkannya; kalau dia PASS, ketuk tombol **PASS / LEWAT** di kartu lawan tersebut.
1. Baca panel **Hasil Analisis**. Kartu peringkat #1 juga ditandai glow biru di tangan Anda dan siluet biru di papan.

Mode LIVE aktif sejak aplikasi dibuka, jadi hasil analisis langsung diperbarui setiap kali Anda menambah informasi — tidak ada tombol "hitung" yang harus ditekan.

> Catatan penting: aplikasi tidak tahu kartu lawan. Yang ia lakukan adalah menghitung **peluang** berdasarkan informasi yang Anda berikan. Semakin rajin Anda mencatat PASS dan atribusi kartu, semakin tajam hasilnya.

1.5 Cara membaca buku ini

Buku ini disusun sepuluh bagian, dari yang paling praktis sampai yang paling teknis.

Bagian	Isi	Cocok dibaca oleh
I	Pengenalan, masalah, fitur, cara pakai cepat	semua pembaca
II	Dasar permainan domino dan istilah meja	pemain baru
III	Instalasi, menjalankan, konfigurasi, tunnel, PWA	yang memasang aplikasi
IV	Panduan antarmuka langkah demi langkah	pemakai harian
V	Cara membaca dan memercayai hasil analisis	pemakai serius
VI	Arsitektur dan isi mesin (algoritma, server, frontend)	pengembang
VII	Pengujian, pemeriksaan mandiri, dan CI	pengembang
VIII	Panduan strategi bermain	pemain yang ingin naik level
IX	Rujukan API lengkap dengan contoh	integrator/pengembang
X	Pemecahan masalah, FAQ, glosarium, lampiran	semua pembaca

Bila Anda baru pertama kali memakai aplikasi ini, urutan yang disarankan adalah Bagian II, III, IV, lalu VIII. Bila Anda penasaran dengan angka-angkanya, lanjutkan ke Bagian V dan VI.

1.6 Pernyataan jujur tentang batasan

Sebuah buku yang baik tidak menjual alatnya secara berlebihan. Berikut hal-hal yang **tidak** bisa dilakukan aplikasi ini.

- **Aplikasi tidak mengintip kartu lawan.** Semua angka berasal dari simulasi dengan batasan yang Anda masukkan sendiri. Bila Anda salah mencatat, hasilnya salah.
- **Hasil simulasi bukan kepastian.** Win rate 62% berarti: dalam model simulasi, Anda menang pada sekitar 62 dari 100 gim. Sisa 38% adalah kemungkinan kalah.
- **Kualitas rekomendasi bergantung pada jumlah simulasi.** 200 simulasi per kartu memberi gambaran kasar; 2000 sampai 5000 memberi gambaran halus dengan waktu tunggu lebih lama.
- **Aplikasi tidak menggantikan keputusan.** Ia menampilkan peluang; keputusan tetap milik Anda, dan faktor meja (gaya main lawan nyata, kesepakatan lokal, kebiasaan kelompok) tidak seluruhnya bisa dimodelkan.
- **Tidak ada basis data permainan.** Semua data sesi disimpan di peramban Anda sendiri.

Bagian VI dan VII menjelaskan dengan tepat batasan mana yang berasal dari desain dan mana yang berasal dari sifat statistik Monte-Carlo.

BAGIAN II

Dasar Permainan Domino

Bagian ini menjelaskan aturan domino 28 kartu sebagaimana dipahami oleh aplikasi. Tujuannya bukan menggantikan aturan rumah Anda, melainkan menyamakan istilah: apa yang dimaksud "klik", "balak", "ujung", "PASS", "cangkul", dan "adu" di dalam seluruh buku ini dan di seluruh antarmuka aplikasi.

2.1 Kartu: 28 klik dari 0 sampai 6

Satu set domino yang dipakai aplikasi berisi 28 kartu. Setiap kartu punya dua sisi, dan setiap sisi bernilai salah satu angka 0 sampai 6. Setiap pasangan angka muncul tepat sekali, termasuk kartu yang kedua sisinya sama:

- kartu dengan dua angka berbeda: 21 buah, misalnya `2-5`, `0-6`, `3-4`;
- kartu dengan dua angka sama (disebut **balak**): 7 buah, yaitu `0-0`, `1-1`, `2-2`, `3-3`, `4-4`, `5-5`, `6-6`.

Totalnya 21 + 7 = 28 kartu. Karena itulah permainan ini disebut domino 28 kartu.

Nilai mata sebuah kartu adalah jumlah kedua angkanya: `6-6` bernilai 12, `2-5` bernilai 7, `0-0` bernilai 0. Nilai mata dipakai untuk menentukan pemenang ketika permainan berhenti sebelum ada yang habis kartunya (lihat bagian Adu di bawah).

Setiap angka 0 sampai 6 muncul tepat **tujuh kali** di seluruh set. Fakta sederhana ini adalah dasar dari peta angka di aplikasi, dan alasan mengapa panel peta angka selalu menjumlahkan angka yang di tangan Anda, angka yang sudah keluar, dan angka yang belum diketahui menjadi tepat tujuh.

Angka	Kartu yang mengandungnya	Jumlah
0	0-0, 0-1, 0-2, 0-3, 0-4, 0-5, 0-6	7
1	0-1, 1-1, 1-2, 1-3, 1-4, 1-5, 1-6	7
2	0-2, 1-2, 2-2, 2-3, 2-4, 2-5, 2-6	7
3	0-3, 1-3, 2-3, 3-3, 3-4, 3-5, 3-6	7
4	0-4, 1-4, 2-4, 3-4, 4-4, 4-5, 4-6	7
5	0-5, 1-5, 2-5, 3-5, 4-5, 5-5, 5-6	7

6	0-6, 1-6, 2-6, 3-6, 4-6, 5-6, 6-6	7

Lebih dari setengah kartu (4 dari 7) mengandung angka 4, 5, atau 6. Ini sebabnya angka besar cenderung "berat" dan mengapa membuang balak besar lebih awal sering merupakan langkah yang baik.

2.2 Meja: rantai dua ujung

Kartu yang sudah dimainkan membentuk **rantai** di meja. Rantai itu punya dua ujung: ujung kiri dan ujung kanan. Sebuah kartu hanya boleh dimainkan bila salah satu angkanya sama dengan salah satu angka di ujung rantai. Setelah dimainkan, angka yang tidak dipakai untuk menyambung menjadi ujung baru di sisi itu.

Contoh sederhana. Misalkan rantai saat ini:

```
[4|2] [2|6]       ujung kiri = 4, ujung kanan = 6
```

Kartu `6-1`, `6-3`, atau `6-6` bisa dimainkan ke ujung kanan (menyambung angka 6). Kartu `4-0` atau `4-3` bisa dimainkan ke ujung kiri (menyambung angka 4). Balak `2-2` tidak bisa dimainkan sekarang, karena angka 2 tidak ada di ujung mana pun.

Bila sebuah kartu cocok di **kedua** ujung sekaligus (misalnya di rantai di atas kartu `4-6`), aplikasi tidak memilih sendiri. Muncul jendela **Pilih Ujung Papan** yang menampilkan akibat masing-masing pilihan: "Kiri jadi X" dan "Kanan jadi Y". Ini penting, karena arah penyambungan sering menjadi inti strategi.

Kartu pertama. Pada giliran pertama, meja masih kosong. Kartu apa pun boleh dimainkan; kartu itu menjadi pusat rantai, dan kedua angkanya menjadi ujung kiri dan ujung kanan sekaligus. Di papan aplikasi, kartu pertama ini diberi penanda emas dan digambar **berdiri tegak** agar mudah ditemukan.

2.3 PASS: ketika tidak bisa jalan

Bila tidak ada kartu di tangan yang bisa menyambung ke salah satu ujung, pemain harus **PASS** (lewat). Bila semua pemain dalam satu putaran penuh PASS, permainan berhenti karena **macet** (adu). Bila tidak ada yang habis dan permainan berhenti, pemenangnya ditentukan dengan membandingkan nilai mata (lihat bagian Adu).

Untuk aplikasi, PASS lawan bukan sekadar catatan administratif — ini **bukti**. Bila lawan melewati giliran saat ujung meja adalah angka 3 dan 1, maka dipastikan lawan tersebut tidak memegang kartu berangka 3 **dan** tidak memegang kartu berangka 1. Karena setiap angka hanya ada tujuh kartu, satu kejadian PASS bisa menghapus banyak kemungkinan dari perhitungan. Bab tersendiri di Bagian IV menjelaskan cara mencatatnya.

2.4 Cangkul: menyambar dari sisa

Pada banyak kelompok, kartu yang tidak dibagikan tetap tersedia di tengah meja dan boleh disambar ketika pemain tidak bisa jalan. Mekanik ini disebut **cangkul**. Pemain yang PASS mengambil kartu dari sisa sampai menemukan kartu yang bisa dimainkan, atau sampai sisanya habis.

Jumlah kartu sisa ditentukan oleh pembagian: 28 kartu dikurangi (jumlah pemain x jumlah kartu per pemain).

Pemain	Kartu/pemain	Dibagikan	Sisa (cangkul)
2	7	14	14
3	7	21	7
4	7	28	0
4	6	24	4
3	5	15	13

Bila tidak ada sisa (misalnya 4 pemain x 7 kartu), permainan berjalan tanpa cangkul. Aplikasi mendukung dua perilaku yang bisa dipilih di panel Setup:

- **Ya — sisa bisa disambar**: pemain yang PASS menarik kartu sisa dan kartu itu masuk kembali ke perhitungan.
- **Tidak — sisa mati**: kartu sisa tidak pernah masuk permainan. Aplikasi lalu menghitung **kartu sisa mana yang paling mungkin mati** dan menampilkannya di panel Kartu Sisa, dengan warna berbeda sesuai tingkat kemungkinannya.

2.5 Adu: ketika meja macet

Adu (deadlock) terjadi ketika permainan berhenti karena tidak ada lagi langkah yang bisa diambil semua pemain, dan tidak ada kartu sisa untuk disambar. Tidak ada yang habis kartunya, sehingga pemenang ditentukan dari nilai mata.

Aplikasi menyediakan dua konvensi pembandingan, dan keduanya umum dipakai berbeda-beda di lapangan:

Konvensi	Aturan	Catatan pemakaian
Mata terkecil	nilai mata Anda dibandingkan dengan nilai mata terkecil di antara lawan; lebih kecil menang	umum di banyak kelompok gaple; hukuman untuk tangan berat
Rata-rata	nilai mata Anda dibandingkan dengan rata-rata nilai mata semua lawan	lebih toleran bila ada satu lawan yang tersisa sangat berat

Selain itu ada pilihan perilaku saat **seri** (nilai mata sama dengan pembanding): dihitung **menang** atau **kalah**. Pilihan ini terlihat sepele, tetapi mengubah perhitungan secara nyata, dan aplikasi memakainya secara konsisten di dalam simulasi maupun di laporan "menang lewat adu" pada panel hasil.

> Mengapa konvensi ini penting? Bila aturan meja Anda adalah "terkecil menang" tetapi aplikasi disetel "rata-rata", seluruh angka peluang menang akan menyimpang, dan rekomendasi kartu bisa berubah urutan. Setel konvensi meja **sebelum** mulai mencatat.

2.6 Jumlah pemain dan arah giliran

Aplikasi mendukung 2, 3, dan 4 pemain. Aturan kartu per pemain mengikuti pembagian yang wajar: maksimum 14 kartu untuk 2 pemain, 9 kartu untuk 3 pemain, dan 7 kartu untuk 4 pemain (aplikasi membulatkan ke bawah: 28 dibagi jumlah pemain).

Yang sering dilupakan: **urutan giliran memengaruhi hasil**. Bila Anda bermain dan lawan yang bermain tepat setelah Anda adalah lawan yang menahan angka tertentu, hasilnya berbeda dibandingkan bila yang bermain setelah Anda adalah lawan yang lemah di angka itu. Karena itu aplikasi punya pilihan **Main Setelah Saya** di panel Setup. Pilihan ini dipakai dua kali oleh mesin:

1. di dalam simulasi, urutan giliran disusun sesuai pilihan tersebut;
2. pada mode **teliti**, lawan yang bermain tepat setelah Anda-lah yang diberi permainan ketat (tanpa derau), karena dialah yang paling menentukan kelanjutan langkah Anda.

2.7 Istilah ringkas

Istilah	Arti di aplikasi ini
Klik / kartu	satu keping domino dengan dua angka, ditulis a-b
Balak	kartu dengan dua angka sama, misalnya 5-5
Nilai mata	jumlah kedua angka sebuah kartu atau sebuah tangan
Ujung	angka bebas di kiri dan/atau kanan rantai meja
PASS / lewat	tidak ada kartu yang bisa dimainkan
Cangkul	menyambar kartu sisa setelah PASS
Sisa mati	kartu sisa yang tidak pernah masuk permainan
Adu	permainan berhenti, pemenang ditentukan nilai mata
Dominasi	angka yang Anda kuasai (muncul 3 kali atau lebih di tangan)
Jerat	menyisakan angka dominan di ujung agar lawan terpaksa membuka jalan Anda
Kunci	meninggalkan ujung yang hanya Anda miliki aksesnya
Atribusi	menandai kartu di papan sebagai milik lawan tertentu
Win rate	persentase kemenangan dari simulasi untuk sebuah langkah
Interval keyakinan	rentang tempat nilai sebenarnya kemungkinan besar berada

Bagian III akan menuntun Anda memasang dan menjalankan aplikasi.

BAGIAN III

Memasang & Menjalankan

III

Bagian ini menuntun Anda dari komputer kosong sampai aplikasi berjalan di peramban. Semuanya berjalan **lokal***: tidak ada akun, tidak ada pendaftaran, tidak ada data yang dikirim ke luar komputer Anda.*

3.1 Kebutuhan minimum

Komponen	Kebutuhan	Catatan
Python	3.10 atau lebih baru (disarankan 3.11+)	dipakai untuk server, build, dan pengujian
Compiler C++20	MSVC (Windows), g++, atau clang++	hanya untuk membangun mesin analisis
Peramban modern	Chrome, Edge, Firefox, atau Safari terbaru	mendukung PWA dan service worker
Ruang disk	sekitar 5 MB untuk kode + 1 MB biner mesin	tanpa basis data

Anda **tidak** butuh Node.js, npm, Docker, atau koneksi internet setelah pemasangan awal. Antarmuka memakai HTML, CSS, dan JavaScript biasa tanpa kerangka kerja, dan mesin analisisnya adalah satu berkas C++ mandiri tanpa pustaka pihak ketiga.

Pemeriksaan cepat sebelum mulai, di jendela terminal:

```
python --version
```

Bila perintah itu gagal, pasang Python dari python.org dan pastikan opsi "Add Python to PATH" dicentang pada saat pemasangan di Windows.

3.2 Menjalankan dalam satu perintah

Cara terpendek dan paling disarankan:

```
python start.py
```

Perintah itu menjalankan empat langkah berurutan:

1. **Membangun mesin** lewat `tools/build_engine.py`. Langkah ini pintar: bila biner

sudah ada dan lebih baru daripada sumber `engine/domino_engine.cpp`, pembangunan dilewati sama sekali.

1. **Memeriksa mesin** lewat `tools/selftest.py`. Mesin menjalankan sebelas pemeriksaan mandiri (lihat Bagian VII). Bila ada satu saja yang gagal, aplikasi menolak berjalan, karena angka analisis tidak bisa dipercaya.
1. **Memeriksa port.** Bila port 2024 sudah melayani `/api/health`, start.py berhenti dengan pesan bahwa server lain mungkin sedang berjalan.
1. **Menyalakan server** di `http://127.0.0.1:2024`.

Keluaran yang wajar terlihat seperti ini:

```
[build] engine up to date: C:\...\bin\domino_engine.exe
[selftest] [PASS] deck_28_kartu - 28 tile unik 0-0 .. 6-6
[selftest] [PASS] parser_json - ok
...
[selftest] semua check LOLOS
[server] Domino Analyzer Pro v3 -> http://127.0.0.1:2024
[server] engine: C:\...\bin\domino_engine.exe (OK)
```

Setelah itu buka `http://127.0.0.1:2024` di peramban. Server berjalan di latar depan; menutup jendela terminal akan mematikannya.

Untuk memeriksa pemasangan **tanpa** menyalakan server:

```
python start.py --check
```

Perintah ini hanya membangun mesin dan menjalankan pemeriksaan mandiri. Berguna untuk memastikan masalah ada di aplikasi, bukan di jaringan atau port.

3.3 Membangun mesin secara manual

`tools/build_engine.py` mencari compiler dengan urutan berikut:

1. **MSVC** di Windows, ditemukan lewat `vswhere.exe`, lalu lingkungannya dimuat dari `vcvars64.bat`. Hasilnya `bin/domino_engine.exe`.
1. **g++** bila ada di PATH. Hasilnya `bin/domino_engine`.
2. **clang++** bila ada di PATH.

Pilihan tambahannya:

```
python tools/build_engine.py           # build bila sumber lebih baru
python tools/build_engine.py --force    # selalu build ulang
python tools/build_engine.py --debug    # /Od /Zi atau -O0 -g, untuk penelusuran
```

Bendera yang dipakai: `/EHsc /std:c++20 /utf-8 /O2 /DNDEBUG` untuk MSVC, dan `-std=c++20 -O2 -DNDEBUG -Wall -Wextra` untuk g++/clang++.

Bila tidak ada compiler yang ditemukan, skrip mencetak `TIDAK ADA compiler yang bisa dipakai (MSVC/g++/clang++)` dan keluar dengan kode 1. Aplikasi web tetap bisa dibuka, tetapi setiap analisis akan dijawab dengan pesan bahwa mesin belum dibangun (HTTP 503) — jadi pemasangan belum lengkap sampai langkah ini berhasil.

> Untuk Windows, cara termudah mendapat MSVC adalah memasang **Visual Studio Build Tools** dan mencentang beban kerja "Desktop development with C++". Setelah itu build berjalan tanpa perlu membuka Visual Studio.

3.4 Menjalankan server secara terpisah

```
python server/domino_server.py
```

Perintah ini berguna bila Anda ingin mengubah kode frontend tanpa membangun ulang mesin, atau saat melakukan penelusuran masalah. Berkas statis dilayani apa adanya dengan `Cache-Control: no-cache`, jadi menyegarkan peramban cukup untuk melihat perubahan HTML, CSS, atau JavaScript.

Peringatan yang mungkin muncul:

Pesan	Arti	Tindakan
PERINGATAN: engine C++ belum ada	biner mesin belum dibangun	jalankan python tools/build_engine.py
mode proteksi AKTIF: POST /api/* butuh header X-App-Token	APP_TOKEN terisi	kirim header itu dari klien, atau kosongkan variabelnya

3.5 Konfigurasi lewat variabel lingkungan

Variabel	Bawaan	Arti
HOST	127.0.0.1	alamat bind server. Hanya komputer ini yang bisa mengakses bila 127.0.0.1
DOMINO_PORT	2024	port server. Nilai <= 0 dikembalikan ke 2024
APP_TOKEN	(kosong)	bila diisi, semua POST /api/* wajib menyertakan header X-App-Token
DOMINO_THREADS	semua inti logis (maks 32)	jumlah thread mesin C++ untuk menjalankan simulasi. 1 = sekuensial. Hasilnya identik pada jumlah thread berapa pun
DOMINO_MAX_ENGINE	1	berapa proses mesin boleh berjalan bersamaan. 1 = permintaan diantrekan supaya tiap analisis memakai seluruh inti CPU
DOMINO_ENGINE_WAIT	60	batas tunggu (detik) dalam antrean mesin sebelum server menjawab 503

Contoh menyalakan di seluruh antarmuka jaringan lokal:

```
DOMINO_PORT=8080 HOST=0.0.0.0 python server/domino_server.py
```

Contoh menahan mesin agar memakai 4 thread saja (komputer dipakai kerja lain):

```
DOMINO_THREADS=4 python start.py
```

> **Catatan akurasi.** Berkas README proyek menyebut variabel `PORT`, tetapi kode di `server/domino_server.py` membaca `DOMINO_PORT` saja. Selain itu, `start.py` memeriksa ketersediaan port pada angka tetap 2024. Jadi bila Anda mengubah port lewat `DOMINO_PORT`, jalankan server secara manual — jangan lewat `start.py`.

> **Peringatan keamanan.** `HOST=0.0.0.0` membuat aplikasi bisa diakses siapa pun di jaringan yang sama, dan setiap permintaan analisis membebani CPU komputer Anda. Endpoint analisis memang dibatasi (maksimum 1.700.000 simulasi per permintaan), tetapi tetap lebih aman membiarkannya di `127.0.0.1` kecuali Anda memang membutuhkan akses dari perangkat lain.

3.5.1 Tentang APP_TOKEN dan antarmuka web

`APP_TOKEN` adalah pengaman untuk klien luar: skrip, `curl`, atau alat monitoring. Bila variabel itu diisi, klien harus mengirim header `X-App-Token` yang nilainya sama.

Penting diketahui: **antarmuka web bawaan belum mengirim header itu.** Bila Anda menyalakan `APP_TOKEN`, panel analisis di peramban akan menerima HTTP 401 dan menampilkan pesan kesalahan. Jadi ada dua pilihan yang masuk akal:

- biarkan `APP_TOKEN` kosong dan lindungi aplikasi di lapisan lain (misalnya Cloudflare Access atau aturan firewall), atau
- isi `APP_TOKEN` bila Anda hanya memakai API dari skrip, dan jangan buka antarmuka web ke publik.

3.6 Membuka akses dari internet (opsional)

Aplikasi ini dirancang untuk pemakaian lokal. Namun ada kalanya Anda ingin memantaunya dari ponsel saat bermain bersama orang lain. Pola paling aman adalah **tunnel terowongan yang memberi nama acak**, bukan membuka port ke internet.

```
# jendela 1
python start.py
# jendela 2
cloudflared tunnel --url http://127.0.0.1:2024
```

`cloudflared` akan mencetak URL publik acak seperti `https://kata-acak.trycloudflare.com`. URL itu berlaku selama prosesnya hidup dan berhenti bekerja begitu Anda menutupnya.

Yang perlu Anda sadari sebelum melakukan ini:

- **URL itu terbuka untuk siapa saja yang tahu alamatnya.** Tidak ada kata sandi.
- **Setiap permintaan analisis memakai CPU Anda.** Batas 1.700.000 simulasi per permintaan ada tepat untuk alasan ini, tetapi beberapa pengunjung sekaligus tetap bisa membuat komputer Anda sibuk.
- **Isi sesi permainan bersifat pribadi.** Tidak ada data yang dikirim keluar, tetapi siapa pun yang membuka URL itu bisa memakai analisis Anda.

Karena itu, untuk pemakaian publik, jauh lebih baik memasang pengaman di lapisan tunnel (Cloudflare Access, atau proxy yang meminta kata sandi) daripada menambah kerumitan di dalam aplikasi.

3.7 Memasang sebagai aplikasi (PWA)

Aplikasi ini menyertakan manifest dan service worker, sehingga bisa dipasang seperti aplikasi biasa. Manfaatnya: jendela sendiri tanpa bilah alamat, dan antarmuka tetap membuka saat koneksi ke server terputus.

Di Android (Chrome). Buka `http://127.0.0.1:2024` (atau alamat LAN Anda) → menu tiga titik → **Tambahkan ke layar utama**.

Di iOS (Safari). Buka alamatnya → tombol **Bagikan** → **Tambahkan ke Layar Utama**.

Di desktop (Chrome/Edge). Klik ikon pasang di bilah alamat, atau menu → **Pasang aplikasi**.

Yang perlu dipahami: PWA memasang **antarmuka**-nya supaya bisa dibuka offline. Analisis tetap membutuhkan server dan mesin yang berjalan, karena perhitungan beratnya memang dilakukan di komputer, bukan di peramban. Saat offline, layar terakhir yang tersimpan di tembolok masih bisa dibuka, tetapi tombol analisis akan gagal terhubung.

Bila Anda mengubah berkas di `web/`, naikkan nama tembolok di `web/sw.js` (saat ini `domino-analyzer-v3.19`) agar peramban mengambil versi baru. Ini praktik yang disengaja: strategi temboloknya adalah *cache-first* untuk berkas statis, jadi perubahan tidak langsung terlihat tanpa mengganti nama tembolok.

3.8 Struktur proyek

```
domino move analisis/
├── start.py                  peluncur: build + selftest + serve
├── engine/
│   └── domino_engine.cpp     mesin analisis C++20 (satu berkas, tanpa dependensi)
├── bin/
│   ├── domino_engine.exe     biner mesin (Windows)
│   └── domino_engine         biner mesin (Linux/macOS)
├── server/
│   └── domino_server.py      server HTTP + API + penyaji berkas statis
├── web/
│   ├── index.html            kerangka antarmuka
│   ├── app.js                seluruh logika antarmuka
│   ├── styles.css            tema gelap dan tata letak responsif
│   ├── sw.js                 service worker (tembolok offline)
│   ├── manifest.json         metadata PWA
│   └── icon.svg              ikon aplikasi
├── tools/
│   ├── build_engine.py       pencari compiler + pembangun mesin
│   ├── selftest.py           menjalankan pemeriksaan mandiri mesin
│   ├── test_server.py        uji unit server (29 tes)
│   ├── test_engine_parallel.py  uji paralel/determinisme mesin (beban penuh: DOMINO_SLOW_TESTS=1)
│   ├── test_ui_heavy.py      uji antarmuka tanpa kepala untuk tier 1,7 juta (perlu playwright)
│   ├── build_baseline.py     bangun mesin dari git HEAD + rekam hasil golden
│   ├── audit_golden.py       bandingkan keluaran mesin byte-per-byte dengan HEAD
│   ├── bench_engine.py       ukur kecepatan: HEAD vs 1 thread vs paralel
│   ├── build_docs.py         pembangun buku dokumentasi ini (Markdown, PDF, DOCX)
│   └── build_deck.py         pembangun presentasi PPTX
├── docs/                     buku dokumentasi (sumber + hasil)
└── .github/workflows/ci.yml  alur integrasi berkelanjutan
```

Susunan ini mengikuti satu aturan: **antarmuka boleh sederhana, mesin harus benar.** Karena itu mesin analisis berdiri sendiri sebagai satu berkas C++ yang bisa dijalankan dari terminal tanpa server sama sekali, seperti yang akan Anda lihat di Bagian VI.

3.9 Menguji pemasangan Anda

Lima perintah yang membuktikan pemasangan berjalan benar:

```
python tools/selftest.py      # 11 pemeriksaan mandiri mesin
python tools/test_server.py   # 29 uji unit server (butuh mesin untuk tes end-to-end)
python tools/test_engine_parallel.py   # 8 uji paralel/determinisme (4 di antaranya beban penuh, pe
rlu DOMINO_SLOW_TESTS=1)
python tools/test_ui_heavy.py # 6 uji antarmuka tanpa kepala untuk tier 1,7 juta
                              #   (perlu `pip install playwright`; dilewati bila tak ada peramban)
python start.py --check       # build + selftest saja, tanpa menjalankan server
```

Bila semuanya keluar dengan kode 0 dan mencetak "OK" atau "semua check LOLOS", pemasangan Anda sudah lengkap. Bagian VII menjelaskan apa yang sebenarnya diperiksa oleh masing-masing perintah.

BAGIAN IV

Panduan Antarmuka

IV

Bagian ini menjelaskan setiap bagian layar, apa yang dilakukannya, dan bagaimana memakainya dalam permainan nyata. Urutannya sengaja mengikuti alur kerja pemain: menyiapkan meja, mengisi kartu, mencatat apa yang terjadi, lalu membaca hasil.

4.1 Tata letak layar

Antarmuka menyesuaikan diri dengan lebar layar, dan ada tiga susunan yang berbeda.

Ponsel (lebar di bawah 1024px). Semua panel ditumpuk dalam satu kolom dengan urutan: judul, setup, bilah aksi, bilah kemajuan, papan, tangan Anda, hasil analisis, lawan, peta angka, lalu kartu sisa.

Desktop (1024px ke atas). Tampilan berubah menjadi tiga kolom:

Kolom	Lebar	Isi
Kiri	300px	setup, banner kualitas, bilah aksi, bilah kemajuan
Tengah	sisa lebar	papan, kartu tangan, hasil analisis
Kanan	380px	lawan, peta angka, kartu sisa

Layar lebar (1500px ke atas). Kolom kiri dan kanan sedikit dilebarkan (320px dan 420px) agar panel lawan dan hasil lebih lega, dengan lebar maksimum keseluruhan 1560px.

Perubahan tata letak ini murni tampilan: semua perhitungan dan data sesi sama sekali tidak berubah, dan lebar panel dihitung ulang secara otomatis oleh pengamat ukuran (`ResizeObserver`) sehingga papan tetap tersusun rapi saat jendela diubah ukurannya.

4.2 Bilah judul

Di bagian atas terdapat nama aplikasi, daftar kemampuan singkat, penanda versi, dan **penanda status analisis**:

Tampilan	Arti
LIVE dengan titik merah menyala	mode hidup aktif; analisis berjalan otomatis setiap kali data berubah

LIVE dengan ikon jeda	mode hidup dijeda; analisis hanya berjalan bila Anda menekan tombolnya

Mode hidup **aktif sejak aplikasi dibuka**. Banyak pemakai mengira tombolnya rusak karena tidak perlu ditekan; tidak, memang itulah perilaku yang disengaja sejak versi 3.2.

4.3 Panel Setup: memberi tahu aturan meja

Panel ini adalah tempat Anda menyesuaikan aplikasi dengan permainan yang sedang berlangsung. Semua pilihan di sini memengaruhi perhitungan, jadi setel dulu sebelum mencatat kartu.

Kendali	Pilihan	Bawaan	Pengaruh
Pemain	2, 3, 4	4	menentukan jumlah lawan, pembagian kartu, dan urutan giliran
Kartu/Pemain	3 sampai 28 dibagi pemain	maksimum	menentukan jumlah kartu tersembunyi dan kartu sisa
Simulasi/Kartu	1000, 5000, 10000, 50000, 1.700.000	10000	seberapa banyak gim yang disimulasikan untuk setiap kandidat langkah; 50.000 ≈ 95% ±0,4%; 1.700.000 ≈ 95% ±0,1% (paling akurat, paling lambat — lihat peringatan di bawah)
Main Setelah Saya	Lawan 1 sampai Lawan n	Lawan 1	siapa yang bermain tepat setelah Anda; juga menentukan lawan mana yang bersikap ketat di mode teliti
Adu: Banding	Mata terkecil / Rata-rata	Mata terkecil	cara menentukan pemenang saat meja macet
Seri = ?	Menang / Kalah	Menang	perilaku bila nilai mata Anda sama dengan pembanding
Main Cangkul?	Ya / Tidak	Ya	apakah kartu sisa boleh disambar atau mati
Lawan respons cerdas	aktif/tidak	tidak aktif	mengaktifkan mode teliti (lebih akurat, lebih lambat)

Di bawah kendali-kendali itu ada satu baris keterangan yang menghitung sendiri hasil pilihan Anda, misalnya:

```
4 pemain × 7 kartu = 28 dibagikan | Semua kartu terpakai (tanpa sisa) (maks 7/pemain)
3 pemain × 5 kartu = 15 dibagikan | Cangkul: 13 kartu (aktif) (maks 9/pemain)
```

Baris ini sengaja dibuat: kesalahan paling sering terjadi adalah salah mengisi jumlah kartu per pemain, dan angka sisa yang salah membuat seluruh perhitungan menyimpang.

Berapa jumlah simulasi yang sebaiknya dipakai? Tabel berikut berasal dari sifat statistiknya, bukan dari tebakan:

Simulasi	Waktu tunggu (perkiraan)	Kapan dipakai
1000	paling cepat	saat bermain cepat atau di ponsel lama
5000	cepat	pemakaian sehari-hari

10000 (bawaan)	sedang	keseimbangan yang baik
50000	hitungan detik	keputusan besar, atau saat Anda ingin membaca selisih tipis
1.700.000	puluhan detik (peringatan aktif)	keputusan paling penting; interval keyakinan menyempit sampai ±0,1% sehingga selisih kecil antar langkah benar-benar bisa dipercaya

Semakin banyak simulasi, semakin sempit interval keyakinan, sehingga perbedaan kecil antar langkah bisa Anda percayai. Sebaliknya, pada 1000 simulasi, selisih dua persen tidak berarti apa-apa. Bab tentang interval keyakinan di Bagian V menjelaskan hal ini dengan angka.

4.3.1 Tier 1.700.000 simulasi: peringatan dan aturan mainnya

Tier tertinggi memberi interval paling sempit (≈95% ±0,1%), tetapi satu analisis memakai **seluruh inti CPU selama puluhan detik**. Empat hal berikut menjaganya tetap nyaman:

Perilaku	Penjelasan
Konfirmasi sebelum dipakai	saat Anda memilih 1.700.000, aplikasi menampilkan peringatan berisi perkiraan waktunya. Menolak akan mengembalikan pilihan sebelumnya
Peringatan tetap terlihat	selama tier ini aktif, banner kuning di bawah panel Setup menyebutkan perkiraan waktu untuk posisi Anda saat ini (dihitung dari jumlah kandidat langkah × jumlah simulasi × jumlah inti komputer)
Mode Live dimatikan	analisis tidak dijalankan otomatis setiap kali Anda menempatkan kartu. Tanpa aturan ini, setiap langkah memicu 1,7 juta simulasi baru dan CPU tidak pernah berhenti
Tombol Batal benar-benar menghentikan	menekan Batal saat analisis berjalan tidak hanya membatalkan tampilan hasil — server menghentikan proses mesinnya, sehingga CPU langsung bebas
Label tombol tidak menipu	teks tombol selalu mengikuti tier yang sedang dipilih: memilih tier berat mengubahnya menjadi "Analisis 1,7 jt (sekali)", dan menurunkannya kembali mengembalikannya ke "Analisis Live"
Live dihidupkan lagi hanya kalau tadi ia yang mematikannya	menurunkan tier dari 1,7 juta menyalakan Live kembali otomatis; tetapi bila Anda sendiri yang menjeda Live, ia tetap jeda — aplikasi tidak mengambil alih keputusan Anda
Sesi yang diimpor ikut diperingatkan	memuat sesi lama yang tersimpan pada tier 1,7 juta langsung menampilkan banner peringatan dan label sekali-jalan, bukan diam-diam masuk tier berat

Selama analisis berjalan, bilah kemajuan menampilkan waktu berjalan dan perkiraan sisa. Perkiraannya kasar dan sengaja tidak pernah menunjukkan "hampir selesai" sebelum benar- benar selesai.

Waktu nyata di mesin uji (6 inti fisik / 12 utas logis; 4 pemain, cangkul aktif):

Situasi	Kandidat langkah	Waktu
Pertengahan permainan	3	≈7,5 detik

Papan terbuka, banyak pilihan	12	≈30 detik
Mode teliti	3	≈7,6 detik
Papan kosong, seluruh 7 kartu bisa dibuka	7	22 detik (terukur)

Tier ini tetap jauh di bawah batas waktu server (300 detik), jadi hasilnya tidak pernah dipotong di tengah jalan. Yang tidak kalah penting: selagi mesin memakai seluruh 12 utas, antarmuka tetap lancar — diukur 59 frame/detik dan interaksi selesai dalam 18 ms — dan proses mesinnya benar-benar dimatikan dalam ~0,4 detik setelah Anda menekan Batal (diukur ulang oleh uji antarmuka otomatis, lihat Bagian VII).

4.4 Bilah aksi

Tombol	Fungsi
Analisis Live	menyalakan atau menjeda mode analisis otomatis. Saat berjalan, teksnya berubah menjadi "Jeda Live"
Undo	membatalkan satu langkah terakhir (kartu, PASS lawan, atau atribusi)
Export	mengunduh seluruh sesi sebagai satu berkas JSON
Import	memuat sesi dari berkas JSON
Reset	mengosongkan tangan, papan, dan seluruh catatan lawan (mode hidup tetap aktif)

Di bawah bilah aksi terdapat **bilah kemajuan**. Saat analisis berjalan, bilah itu menampilkan persentase, judul pekerjaan, dan rincian singkat, misalnya:

```
30%  Menganalisis...
5 kartu x 2000 sim (engine C++ v3) - data PASS - atribusi lawan - mode teliti
```

Rincian ini berguna untuk memastikan permintaan yang Anda kirim benar-benar memuat data yang Anda harapkan. Bila tertulis "tanpa data PASS" padahal Anda sudah menandai PASS, berarti penandaan itu belum tersimpan.

4.5 Papan: meja domino

Ini bagian yang paling banyak berubah dibandingkan versi awal aplikasi, dan ada beberapa hal yang perlu dipahami supaya tidak bingung.

Penanda ujung. Di atas papan terdapat dua label: "Kiri: X" dan "Kanan: Y". Bila meja masih kosong, keduanya menampilkan tanda minus. Label ini adalah rujukan tercepat saat Anda memutuskan arah.

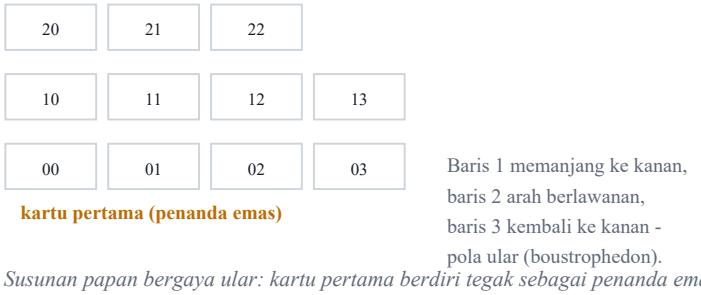
Kartu sisa. Chip kecil di judul papan (ikon kotak diikuti angka) menunjukkan jumlah kartu sisa/cangkul berdasarkan pembagian yang Anda pilih.

Kartu pertama selalu berdiri tegak dan berwarna emas. Kartu pertama yang keluar adalah pusat rantai, dan justru karena itulah ia diberi penanda khusus. Sesudahnya, rantai tumbuh dari kartu itu ke dua arah.

Susunan ular. Rantai tidak memanjang satu baris sampai keluar layar. Papan menyusun kartu dalam baris-baris yang saling berganti arah — memanjang ke kanan, lalu baris berikutnya ke kiri, seperti

ular. Baris pertama (kartu paling awal) ada di bawah, dan rantai tumbuh ke atas. Kartu siku di ujung baris digambar dalam posisi berdiri agar tikungannya tersambung rapi, dan baris ganjil dibalik urutannya supaya angka yang bersambung tetap bertumpuk rapat.

```
[baris 3]  <-- arah ke kiri
[baris 2]  --> arah ke kanan
[baris 1]  [kartu pertama berdiri di sini]
```



Gambar 1. Susunan papan bergaya ular: kartu pertama berdiri tegak sebagai penanda emas, tikungan dibentuk kartu siku.

Papan mengecil otomatis. Bila kartu di papan sudah banyak, ukuran kartu dikecilkan secara bertahap (sampai setengah ukuran normal) supaya seluruh rantai tetap terlihat tanpa perlu menggulir. Aplikasi mencoba lebih dulu baris selebar mungkin (kartu paling besar); bila isi papan belum muat, ia memakai baris yang lebih pendek agar kartu pertama bisa kembali ke tengah. Bila isi papan tetap tidak muat bahkan pada ukuran minimum, papan kembali ke mode gulir seperti cara lama — supaya tidak ada kartu yang hilang.

Bila papan kosong. Papan menampilkan pesan "Papan kosong — pilih kartu dari tangan Anda". Bila analisis sudah punya rekomendasi, pesannya berubah menjadi keterangan bahwa siluet biru di bawah adalah kartu rekomendasi.

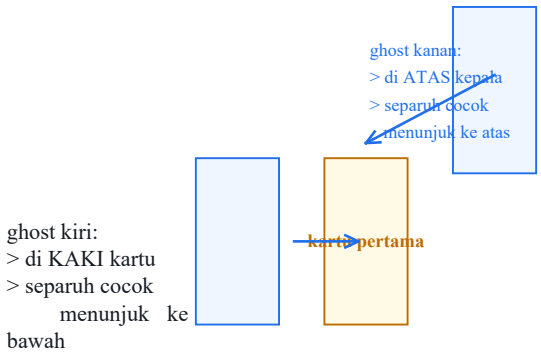
4.6 Tangan Anda

Panel **Kartu Saya** menampilkan hitungan kartu (`5/7 kartu`) dan kepingan kartu Anda.

Tampilan kartu	Arti
Hijau (bisa dimainkan)	ada angka yang cocok dengan salah satu ujung papan sekarang
Abu-abu (tidak bisa dimainkan)	tidak ada angka yang cocok; kartu ini belum bisa keluar
Cincin biru berdenyut	ini kartu peringkat #1 dari analisis terakhir
Semua kartu hijau	papan masih kosong, jadi semua kartu boleh menjadi kartu pertama

Kelima kepingan itu bisa diketuk. Bila kartu hanya cocok di satu ujung, ia langsung dimainkan. Bila cocok di kedua ujung, muncul jendela **Pilih Ujung Papan** yang menampilkan akibat masing-masing pilihan, misalnya "→ Kiri jadi 3" dan "→ Kanan jadi 6".

Di papan, kartu rekomendasi terbaik digambar sebagai **siluet bergaris putus-putus berwarna biru**, ditempatkan relatif terhadap kartu pertama:



Gambar 2. Siluet rekomendasi (ghost) diposisikan relatif ke kartu pertama: di atas kepala saat main ke kanan, di kaki saat main ke kiri.

Hitungan kartu berubah warna menjadi merah bila jumlah kartu tangan Anda melebihi batas kartu per pemain — tanda bahwa ada salah isi yang perlu dibetulkan.

4.7 Memasukkan kartu: modal 28 kartu dan masukan cepat

Tombol + **Pilih Kartu Saya** membuka jendela berisi seluruh 28 kartu, disusun empat kolom. Kartu yang sudah Anda pilih diberi tanda terpilih, dan kartu yang sudah keluar di papan ditampilkan pudar serta tidak bisa dipilih. Menekan kartu yang sudah terpilih akan menghapusnya dari tangan (berfungsi sebagai tombol bolak-balik).

Di atas kisi 28 kartu ada **masukan cepat** berupa dua kotak angka dan tombol "+ Tambah".

Aturan masukan cepat	Perilaku
Isi kotak	satu digit 0 sampai 6; karakter lain dibuang sebelum sempat masuk
Angka 7 ke atas	kotak langsung kosong dan berkedip merah (tidak valid)
Setelah digit pertama	fokus berpindah otomatis ke kotak kedua
Urutan angka	bebas: 5 lalu 4 sama dengan kartu 4-5
Menekan Enter	kartu langsung ditambahkan
Kartu sudah ada di tangan	bukan ditambah, tetapi dihapus (bolak-balik)
Setelah menambahkan	kedua kotak dikosongkan dan fokus kembali ke kotak pertama

Cara ini jauh lebih cepat daripada mengetuk kisi, terutama karena kedua tangan Anda sering masih memegang kartu asli saat bermain.

Batas kartu. Anda tidak bisa menambahkan lebih dari jumlah kartu per pemain; aplikasi menampilkan peringatan bila Anda mencoba.

4.8 Mencatat lawan: panel Lawan

Panel ini adalah jantung akurasi aplikasi. Setiap lawan memiliki satu kartu panel dengan beberapa bagian.

Bagian	Isi	Kegunaan

Nama dan jumlah kartu	mis. "Lawan 2 · 5 kartu"	perkiraan jumlah kartu lawan, dihitung dari kartu yang dibagikan dikurangi kartu Anda dan kartu lawan yang sudah terlihat
PASS / LEWAT	tombol merah	catat bahwa lawan ini tidak bisa jalan
Undo	tombol kecil	membatalkan PASS terakhir lawan ini
PASTI TIDAK PUNYA angka	deretan angka	hasil eliminasi dari semua PASS lawan tersebut
Riwayat pass	mis. `[3	1] → [6
Kartu yang dipastikan dimainkan	mis. `2	5 4
Diduga kuasai	mis. "5 (~1.8 kartu, 35%)"	perkiraan kekayaan angka dari mesin
Kemungkinan kartu	kisi kartu	daftar kartu yang mungkin dipegang lawan ini, bisa diketuk
Filter PASS	mis. "23 dari 26 kartu unknown (tereliminasi 12%)"	berapa banyak kemungkinan yang sudah berhasil dipersempit

Cara mencatat PASS. Ketuk tombol **PASS / LEWAT** pada lawan yang melewati giliran. Aplikasi mencatat ujung meja saat itu (`[kiri|kanan]`), lalu menambahkan kedua angka itu ke daftar "pasti tidak punya" milik lawan tersebut. Tombol ini tidak aktif saat papan masih kosong, karena di awal permainan belum ada ujung untuk diuji.

Cara mencatat kartu lawan (atribusi). Ketuk kartu di panel **Kartu Sisa** atau di kisi **Kemungkinan kartu** milik lawan. Muncul jendela "Siapa yang memainkan kartu ini?" berisi tombol untuk setiap lawan, plus tombol "Tidak tahu / lupa". Bila Anda memilih seorang lawan, kartu itu dinyatakan miliknya dan **dieliminasi dari semua lawan lain** sekaligus. Bila Anda memilih "tidak tahu", kartu tetap tercatat keluar, tetapi tidak diatribusikan kepada siapa pun.

Kisi kemungkinan kartu. Setiap kartu yang mungkin dipegang lawan ditampilkan, dan kartu yang **s aat ini bisa dimainkan** diberi kilau merah. Kilau itu penting: bila lawan menolak jalan padahal kartu itu bisa dimainkan, berarti kartu itu tidak ada di tangannya — dan Anda sebaiknya mencatat PASS untuk mempersempit lebih jauh. Sebaliknya, bila lawan memainkan kartu berkilau merah, kemungkinan besar Anda bisa menebak sisanya.

> Satu aturan penting: jangan menandai PASS berdasarkan tebakan. Data PASS bersifat **bukti keras** dan langsung memaksa simulasi menghormatinya. Satu kesalahan PASS akan menyempitkan kemungkinan kartu lawan secara keliru, dan analisis berikutnya menjadi yakin pada hal yang salah.

4.9 Peta angka

Panel **Peta Angka** menampilkan tujuh baris (angka 0 sampai 6) dengan tiga informasi:

- **Di tangan saya** (bagian biru): berapa kartu berangka itu yang Anda pegang.
- **Sudah keluar** (bagian kuning): berapa yang sudah terlihat di papan.
- **Belum diketahui** (sisa sampai tujuh): berapa yang masih tersembunyi.

Angka yang Anda kuasai tiga kali atau lebih ditandai sebagai **dominan**. Angka yang habis dari peredaran (tidak di tangan Anda, tidak terlihat di papan) ditandai sebagai **exhausted**, artinya angka itu tidak akan pernah muncul lagi. Angka yang sedang berada di ujung papan diberi penanda lokasi.

Peta ini adalah alat penalaran cepat yang paling sering dipakai pemain berpengalaman: bila sebuah angka sudah exhausted, meja akan macet di angka itu, dan pembandingnya adalah nilai mata. Bila angka dominan Anda ada di ujung, Anda sedang mengendalikan permainan.

4.10 Kartu sisa: tiga perilaku berbeda

Panel **Kartu Sisa** berubah perilaku sesuai pengaturan cangkul.

Mode A — semua kartu dibagikan. Bila tidak ada sisa (misalnya 4 pemain x 7 kartu), panel menampilkan keterangan bahwa semua kartu ada di tangan pemain dan Anda diarahkan ke kolom Lawan.

Mode B — cangkul aktif. Judulnya "Kartu Cangkul", dan setiap kepingan bisa diketuk untuk mencatat bahwa **lawan menyambar kartu itu** — persis alur atribusi yang dijelaskan di atas.

Mode C — cangkul mati. Judulnya "Kartu Sisa Mati", dan panel menampilkan perkiraan kartu mana yang paling mungkin tidak pernah masuk permainan, lengkap dengan tingkat kemungkinannya:

Tampilan	Arti
Angka pada chip, mis. "5: ±1.8 mati"	perkiraan berapa kartu berangka 5 yang akan berakhir mati
Kartu berwarna merah tua	kemungkinan mati 50% atau lebih
Kartu berwarna oranye	kemungkinan mati 25% sampai 49%
Kartu biasa	kemungkinan mati di bawah 25%

Perkiraan ini berasal dari simulasi pembagian kartu berulang yang menghormati data PASS dan atribusi Anda. Bila belum ada hasil analisis, panel mengingatkan Anda untuk menjalankan analisis lebih dulu.

4.11 Panel hasil analisis

Panel ini menampilkan seluruh kandidat langkah, diurutkan, beserta alasannya. Strukturnya dijelaskan satu per satu di Bagian V; di sini hanya pengantarnya:

- **Chip ringkasan** di judul: `PASS`, `Atribusi`, `Teliti`, dan `Cangkul n` —

menandakan data apa yang benar-benar dipakai oleh hasil yang sedang tampil.

- **Kotak mode kartu pertama** bila papan masih kosong.
- **Panel Dominasi Angka**: hitungan tujuh angka di tangan Anda.
- **Kartu peringkat** untuk setiap kandidat langkah, masing-masing dengan peringkat

(#1, #2, dan seterusnya), kartu, ujung baru yang dihasilkan, lencana, statistik, dan bar.

- **Kotak rekomendasi strategi** di paling bawah, berisi satu paragraf saran.

Bila tidak ada langkah valid, panel menampilkan panduan PASS: apa yang harus Anda lakukan (menekan tombol PASS pada lawan, atau mencatat PASS Anda sendiri), dan mengapa hal itu membuat analisis berikutnya jauh lebih akurat.

4.12 Banner kualitas kartu

Setelah kartu tangan Anda terisi, muncul banner yang menilai kekuatan kartu Anda:

Nilai	Syarat	Saran yang ditampilkan
SANGAT BAGUS & BERUNTUNG	ada angka yang Anda pegang 5 kali atau lebih	kuasai ujung dengan angka itu
KARTU BAGUS	ada angka yang Anda pegang 4 kali	modal bagus untuk mendominasi
KARTU LUMAYAN	ada angka yang Anda pegang 3 kali	cukup untuk mengontrol satu ujung
KARTU JELEK (BANYAK BALAK)	balak 4 buah atau lebih	buang balak secepatnya, siapkan strategi adu
KARTU STANDAR	selain di atas	buang kartu besar dan balak, siapkan adu

Banner ini penilaian heuristik cepat berdasarkan kartu Anda sendiri; ia tidak memakai simulasi. Angka yang lebih dapat dipercaya tetap berasal dari panel hasil analisis.

4.13 Banner menang

Bila tangan Anda habis sementara sudah ada kartu di papan, muncul banner **TANGAN HABIS — ANDA MENANG**. Ini murni pencatatan pencapaian; aplikasi tidak mengunci apa pun, dan Anda bisa langsung menekan Reset untuk permainan berikutnya.

4.14 Mode hidup, undo, dan pengelolaan sesi

Mode hidup (LIVE). Selama mode hidup aktif, analisis dijalankan ulang secara otomatis setiap kali ada perubahan informasi: menambah atau menghapus kartu tangan, memainkan kartu, mencatat PASS, menghapus PASS, dan bahkan membatalkan langkah lewat Undo. Bila Anda melakukan beberapa aksi berturut-turut sementara analisis sebelumnya masih berjalan, permintaan terbaru **diant rekan**, bukan ditumpuk sampai membebani komputer. Hasil yang tampil selalu berasal dari data paling akhir.

Bila ingin menghentikan perhitungan yang sedang berjalan, tekan tombol yang sama sekali lagi (teksnya "Jeda Live"): analisis yang sedang berjalan dibatalkan, dan bilah kemajuan menampilkan "Live dijeda".

Undo. Setiap aksi yang mengubah keadaan menyimpan salinan keadaan sebelumnya ke sebuah timeline berisi hingga 100 langkah terakhir. Satu kali tekan Undo memulihkan salinan terakhir, sehingga:

- kartu yang Anda mainkan kembali ke tangan,
- kartu lawan yang tercatat kembali menjadi kartu sisa,
- PASS lawan yang terakhir dicatat hilang bersama eliminasi angkanya,
- atribusi kartu ikut dibatalkan.

Undo tidak bisa dipakai saat analisis sedang berjalan, supaya keadaan tidak berubah di tengah perhitungan. Sesudah Undo, analisis dijalankan ulang agar panel hasil tidak menampilkan angka basi.



Setiap aksi menyimpan snapshot state (tangan, papan, atribusi, data PASS) ke timeline; tombol Undo memulihkan snapshot terakhir. Maksimum 100 langkah.

Gambar 3. Timeline undo: kartu, PASS lawan, dan atribusi ikut ter-undo sehingga data pelacakan tidak pernah setengah jalan.

Simpan otomatis. Seluruh sesi disimpan ke penyimpanan lokal peramban setiap kali keadaan berubah, dengan kunci `domino-analyzer-v3`. Menutup tab atau menyegarkan halaman tidak menghilangkan data Anda.

Export dan Import. Tombol Export mengunduh berkas JSON bernama `domino-analisis-TANGGAL.json` berisi pengaturan meja dan seluruh keadaan permainan. Tombol Import membacanya kembali. Ini berguna untuk memindahkan sesi ke perangkat lain atau menyimpan permainan yang menarik untuk dipelajari ulang.

Berkas impor **divalidasi ketat** sebelum diterima: setiap kartu harus berbentuk `angka-angka` dengan angka 0 sampai 6, label pemilik kartu harus `me`, `unknown`, atau `opp1` sampai `opp4`, dan data PASS harus berupa angka yang sah. Berkas yang tidak lolos validasi ditolak dengan pesan "File bukan sesi Domino Analyzer v3 yang valid." Ini bukan sekadar kerapian: berkas impor adalah data dari luar, dan aplikasi tidak boleh begitu saja mempercayainya.

4.15 Pesan kesalahan dan artinya

Pesan di layar	Sebab	Tindakan
"Tidak bisa menghubungi server — apakah python start.py masih berjalan?"	server mati atau port berubah	jalankan ulang server
"Respons server tidak valid (HTTP ...)"	balasan bukan JSON	lihat log server; biasanya kesalahan internal
"engine belum di-build. Jalankan: python tools/build_engine.py"	biner mesin tidak ada (HTTP 503)	bangun mesin
"Kartu ... tidak dikenali"	kunci kartu tidak sah	pilih kartu dari daftar, jangan mengetik manual
"Kartu ... muncul dua kali"	kartu yang sama ada di tangan dan di papan	periksa kembali isian Anda
"Data atribusi tidak konsisten"	satu kartu ditandai milik dua lawan	perbaiki penandaan
"Kartu di tangan melebihi kartu per pemain!"	jumlah kartu tangan melampaui batas	hapus sebagian kartu atau ubah pengaturan

Semua pesan kesalahan dirancang menjelaskan **apa yang harus dilakukan**, bukan menampilkan kode teknis. Bagian IX memuat daftar lengkap kode HTTP yang mungkin muncul bila Anda memakai API secara langsung.

BAGIAN V

Membaca & Memercayai Hasil

Inilah bagian yang membedakan pemakai biasa dari pemakai yang benar-benar diuntungkan aplikasi ini. Angka yang tampil di panel hasil bukan satu nilai tunggal, melainkan lima jenis informasi berbeda yang harus dibaca bersama-sama: peringkat, peluang, selang keyakinan, alasan strategis, dan peringatan.

5.1 Urutan kartu ditentukan mesin, bukan tampilan

Setiap kandidat langkah diberi satu nilai bernama **rankScore** oleh mesin, dan daftar kartu dari mesin sudah dalam urutan menurun berdasarkan nilai itu. Tampilan menambahkan satu penyaringan di atasnya: kartu dengan peluang menang 50% atau lebih dipindahkan ke kelompok **aman** di atas, lalu di dalam tiap kelompok urutannya tetap menurut rankScore.

Praktisnya:

1. Semua kartu dengan win rate 50% atau lebih tampil lebih dulu, diurutkan menurun berdasarkan rankScore.
1. Setelah itu barulah kartu dengan win rate di bawah 50%, juga diurutkan menurun.

Penyaringan ini tidak mengubah peringkat #1 kecuali pada keadaan yang jarang (kartu dengan rankScore tinggi tetapi win rate di bawah 50%). Yang penting bagi Anda: **urutan tampilan adalah rekomendasi akhir aplikasi**, dan angka rankScore ditampilkan agar Anda bisa melihat dasar keputusannya.

```
rankScore = winRate x 100
          + domSupportScore x 0,6
          + trapScore x 0,8
          + blockScore x 0,7
          - selfTrapScore x 0,5
          - riskScore x 0,5
          + deadlockWinRate x 20
          + jumlah blok terjamin x 8
          + rata-rata lawan PASS x 3
          + safetyRatio x 30   (hanya pada mode kartu pertama)
```

Rumus ini bukan skor rahasia; ia empat baris kode di dalam mesin. Yang perlu Anda pahami adalah **bobotnya**, karena bobot itulah yang menjelaskan mengapa suatu langkah menang peringkat.

Komponen	Bobot	Artinya
Peluang menang	100 per 1,0	dasar utama; 1% peluang sama dengan 1 poin
Dukungan dominasi	0,6	ujung cocok dengan angka yang Anda kuasai
Jerat	0,8	menyisakan angka dominan dengan kartu masih di luar
Kunci	0,7	ujung yang tidak bisa diakses lawan
Jebakan diri sendiri	-0,5	menyisakan ujung yang berbahaya bagi Anda
Risiko	-0,5	kartu berbahaya yang sering muncul di simulasi kalah
Menang lewat adu	20 per 1,0	dorongan ke skenario adu
Blok terjamin	8 per lawan	lawan yang pasti mati menurut data PASS
Lawan dipaksa PASS	3	berapa sering lawan lewat setelah langkah ini

Perhatikan bahwa 1 poin rankScore dari peluang menang jauh lebih besar daripada dari komponen mana pun. Mesin tidak membiarkan pertimbangan taktis mengalahkan peluang nyata; komponen lain hanya berfungsi memisahkan langkah-langkah yang peluangnya berdekatan.

5.2 Contoh nyata dengan angka asli

Berikut contoh yang benar-benar dihitung oleh mesin (mesin `domino-cpp/3.0.0`, seed `20240922`, 500 simulasi per kandidat). Situasinya: 4 pemain, 7 kartu per pemain, semua kartu dibagikan habis (tidak ada sisa).

Yang diketahui mesin:

```
tangan saya : 6-6, 3-6, 2-3, 1-5, 4-4
di papan    : 0-3, 0-2, 1-2
milik siapa : 0-3 -> Lawan 1 ; 0-2 -> Lawan 2 ; 1-2 -> Lawan 1
ujung meja  : kiri 3, kanan 1
PASS tercatat: Lawan 1 lewat saat ujung 3 dan 1
giliran     : Lawan 1 bermain setelah saya
aturan      : adu banding mata terkecil, seri dihitung menang, cangkul aktif
```

Hasilnya:

#	Kartu	Ujung baru	Peluang menang	Selang 95%	rankScore	Jerat	Jebakan diri	Lawan PASS
1	3-6 ke kiri	6 | 1	69,2%	65,0 - 73,1%	84,2	15	0	3,08
2	2-3 ke kiri	2 | 1	71,2%	67,1 - 75,0%	60,4	0	6	3,47
3	1-5 ke kanan	3 | 5	60,0%	55,6 - 64,2%	51,3	0	6	2,88

Ini pelajaran paling penting di seluruh buku. Kartu `2-3` punya peluang menang lebih tinggi (71,2% berbanding 69,2%), tetapi aplikasi menempatkan `3-6` di peringkat #1. Alasannya terbaca dari tabel:

- `3-6` memasang ujung 6, dan angka 6 adalah **dominasi** Anda (tangan Anda memuat `6-6`,

`3-6`, dan ujung 6 baru terbentuk). Mesin mencatatnya: "Ujung cocok angka dominan 6 (3 kartu)" dan memberi bonus dukungan dominasi.

- `3-6` sekaligus memasang **jerat**: "JERAT angka 6 terpasang! 4 kartu 6 masih di luar."

Mesin menilai 15 poin untuk jerat itu.

- `2-3` tidak memasang jerat apa pun, dan sebaliknya menyisakan ujung berbahaya bagi diri sendiri (`selfTrapScore` 6).
- Kedua kartu sama-sama memaksa lawan PASS dengan frekuensi tinggi (3,08 dan 3,47 kali per simulasi) — keduanya langkah kuat.

Apakah perbedaan 2 poin persen peluang itu nyata? **Tidak.** Perhatikan selang keyakinannya: 65,0-73,1% dan 67,1-75,0% saling menumpuk hampir seluruhnya. Secara statistik, kedua kartu ini hampir tidak bisa dibedakan dari peluang menangnya saja. Yang membedakan adalah **bentuk permainannya setelah kartu itu dimainkan**: satu memasang jerat dominan, satu menyisakan ujung yang tidak menguntungkan. Inilah alasan rankScore ada, dan alasan Anda sebaiknya membaca alasan strategis, bukan hanya angka peluang.

Cara membaca "4 kartu 6 masih di luar": dari 7 kartu berangka 6 di seluruh set, 3 ada di tangan Anda (`6-6`, `3-6`, dan satu lagi dari kombinasi tangan) dan 4 masih belum terlihat. Selama kartu-kartu itu masih di luar, ujung 6 akan terus "memakan" kartu lawan setiap kali mereka menyambung.

5.3 Kunjungi satu kartu secara lengkap

Bila Anda membuka uraian kartu peringkat #1 pada contoh di atas, inilah isinya.

Lencana: `TERBAIK` (peringkat #1 dengan peluang di atas 50%).

Kotak statistik:

Kotak	Nilai	Arti
Win	69,2%	peluang menang dari simulasi
Jerat	15	kekuatan jerat yang Anda pasang
Kunci	0	tidak ada ujung yang terkunci oleh langkah ini
Lawan Pass	3,1	rata-rata berapa kali lawan lewat dalam satu gim setelah langkah ini
Sisa Mata	6,1	rata-rata nilai mata di tangan Anda saat gim berakhir
Risk	46,6	seberapa sering kartu berbahaya muncul pada gim yang Anda kalahkan

Bar:

- *Peluang Menang* — batang berwarna sesuai tingkat (hijau di atas 50%, kuning 35-50%, merah di bawah 35%), dengan catatan "interval 95%: 65,0-73,1%".
- *Menang lewat ADU* — 25,0% pada contoh ini: seperempat dari kemenangan datang dari skenario meja macet, bukan dari menghabiskan kartu.
- *Kekuatan Jerat* — 15 poin (batang merah muda).

Kotak strategi. Berisi `trapReasons` dan `blockReasons`, yaitu kalimat alasan yang sama seperti yang dipakai mesin untuk menilai langkah. Contohnya pada langkah ini: "JERAT angka 6 terpasang! 4 kartu 6 masih di luar."

Kotak blok terjamin. Muncul hanya bila langkah itu mematikan lawan menurut **bukti** PASS. Kalimatnya berbentuk "Lawan 1 PASTI tidak bisa main (tidak punya 6 maupun 1)!". Pada contoh ini kotak itu kosong, karena lawan yang sudah lewat (Lawan 1) masih punya angka 6 — bukti yang ada hanya menghapus angka 3 dan 1.

Kotak kartu berbahaya. Lima kartu yang paling sering muncul di tangan lawan pada gim yang Anda kalahkan, beserta frekuensinya:

Kartu	Frekuensi	Tafsiran praktis
3-5	96%	hampir selalu ada di tangan lawan saat Anda kalah
5-6	95%	idem
2-5	92%	idem
4-6	92%	idem
2-6	91%	idem

Angka-angka ini jangan dibaca sebagai "kartu itu ada di tangan lawan" — itu tidak benar. Bacalah sebagai: **kartu-kartu inilah yang paling sering menjadi alat kekalahan Anda**, dan itu karena semuanya mengandung angka 5 atau 6, yang memang sedang hidup di meja. Bila frekuensi sebuah kartu mendekati 100% sementara kartu itu masih mungkin dipegang lawan, Anda sebaiknya waspada: kartu itu kemungkinan besar akan keluar.

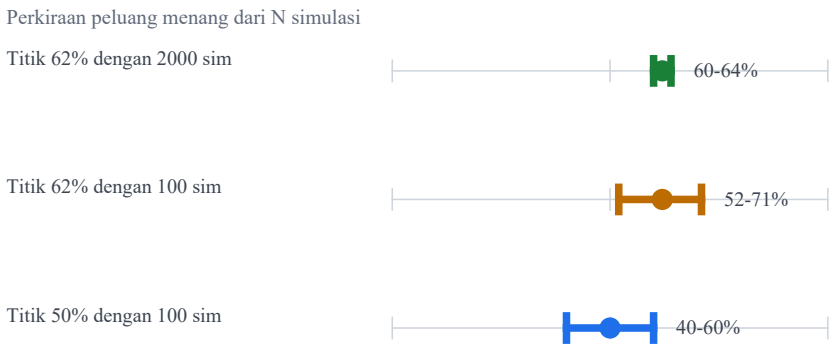
5.4 Interval keyakinan: alat ukur keraguan

Setiap peluang menang disertai **interval keyakinan Wilson 95%** — rentang yang, secara statistik, memuat nilai sebenarnya dengan keyakinan 95% bila simulasi diulang tanpa batas. Rumusnya dihitung mesin:

```
pusat  = (p + z²/2n) / (1 + z²/n)
margin = z x akar( p(1-p)/n + z²/4n² ) / (1 + z²/n)          z = 1,96
selang = pusat ± margin
```

dengan p peluang yang terukur dan n jumlah simulasi. Lebar interval menyempit mengikuti akar jumlah simulasi — inilah alasan menambah simulasi memberi manfaat yang mengecil:

Simulasi	Selang 95% untuk peluang 50%	Lebar
200	43,1% - 56,9%	13,7 poin
500	45,6% - 54,4%	8,7 poin
1000	46,9% - 53,1%	6,2 poin
2000	47,8% - 52,2%	4,4 poin
5000	48,6% - 51,4%	2,8 poin
10000	49,0% - 51,0%	2,0 poin



Gambar 4. Interval Wilson 95%: jumlah simulasi yang lebih banyak membuat selang keyakinan menyempit, sehingga beda tipis tidak menyesatkan.

Cara memakainya: dua kartu hanya benar-benar berbeda bila selangnya tidak saling menumpuk. Pada contoh di atas, 65,0-73,1% dan 67,1-75,0% menumpuk, jadi jangan memperlakukan #1 dan #2 sebagai "yang satu jauh lebih baik". Sebaliknya, kartu ketiga (55,6-64,2%) tidak menumpuk dengan kartu pertama, jadi perbedaan antara #1 dan #3 memang nyata.

5.5 Peringkat mana yang boleh diabaikan

Beberapa keadaan yang sering menyesatkan pemakai baru:

Peluang tinggi tetapi keamanan nol. Di mode kartu pertama, selalu periksa bar **Keamanan Putaran 2**. Bila tertulis 0 kartu bisa dimainkan di putaran kedua, langkah itu berbahaya apa pun peluangnya. Mesin menampilkan peringatan merah tegas untuk kondisi ini, misalnya: "BAHAYA! Tidak ada kartu yang bisa dimainkan di putaran ke-2!"

Kunci melawan peluang. Kadang langkah dengan peluang sedikit lebih rendah tetapi mematikan seorang lawan menurut data PASS akan muncul di #1. Itu bukan kesalahan; itu memang strategi terbaik karena giliran akan kembali ke Anda tanpa lawan bisa mencegahnya.

Ujung yang "mengubah permainan". Bila langkah #1 dan #2 punya selang bertumpuk dan tidak ada jerat atau kunci, pilih yang paling **aman untuk langkah berikutnya** — yaitu yang menyisakan ujung masih hidup untuk Anda sendiri. Bar keamanan putaran kedua bisa Anda pakai sebagai pembanding cepat pada mode kartu pertama; pada mode biasa, perhatikan angka "Jebakan diri" di kotak statistik.

Rekomendasi yang basi. Setiap siluet dan glow rekomendasi divalidasi ulang terhadap keadaan papan saat ini: bila kartu rekomendasi sudah keluar dari tangan Anda atau ujung meja sudah berubah sehingga kartu itu tidak bisa disambungkan, tanda rekomendasi dihilangkan sampai analisis berikutnya selesai. Jadi, glow yang hilang bukan kerusakan — itu pertanda data sudah berubah.

5.6 Profil kekayaan angka lawan

Untuk setiap lawan, mesin menaksir angka mana yang kemungkinan besar ia kuasai, dan seberapa percaya diri taksiran itu. Contoh nyata dari perhitungan di atas:

Lawan	Kartu tersembunyi	Kolam kemungkinan	Angka tersangka	Balak diperkirakan

Lawan 1	6	4 kartu	5 (±6,0 kartu, 100%) dan 4 (±3,0, 50%)	≈ 1,5
Lawan 2	5	20 kartu	5 (±1,8 kartu, 35%) dan 0 (±1,5, 30%)	≈ 1,3
Lawan 3	8	11 kartu	1 (±3,6 kartu, 45%) dan 3 (±3,6, 45%)	≈ 2,2

Cara membacanya: **"kartu tersembunyi"** adalah berapa kartu yang belum terlihat yang diperkirakan ada di tangan lawan itu. **"Kolam kemungkinan"** adalah berapa kartu yang masih mungkin ia pegang sesuai semua batasan (PASS dan atribusi). **Angka tersangka** adalah dua angka teratas dengan perkiraan jumlah kartu dan persentasenya terhadap perkiraan jumlah kartu lawan itu.

Perhatikan Lawan 1 pada tabel: kolamnya hanya 4 kartu, dan taksirannya mencapai 100% untuk angka 5. Ini bukan berarti pasti, tetapi kolamnya memang sudah sangat sempit karena PASS lawan itu menghapus angka 3 dan 1, dan kartu-kartu yang tersisa hampir semuanya berangka 4 atau 5. Semakin banyak PASS yang Anda catat, semakin sempit kolamnya, dan semakin tajam taksiran ini.

5.7 Kartu sisa yang paling mungkin mati

Pada mode cangkul mati, mesin menjalankan simulasi pembagian kartu khusus untuk menghitung **peluang setiap kartu berakhir di sisa** dan **perkiraan jumlah kartu mati per angka**. Contoh keluaran yang bentuknya bisa Anda lihat di panel:

```
Kartu Sisa Mati (4)
  3: ±1,2 mati     5: ±1,8 mati     6: ±1,0 mati
  [kartu berwarna sesuai tingkat: >=50% merah, 25-49% oranye, <25% normal]
```

Manfaat praktisnya: angka yang banyak "mati" di sisa menjadi langka di permainan nyata. Bila 2 dari 7 kartu berangka 5 akan mati, artinya hanya 5 kartu berangka 5 yang beredar — dan permainan lebih muda macet di angka itu.

5.8 Mode kartu pertama: membaca keamanan putaran kedua

Pada giliran pertama, peluang menang saja tidak cukup, karena kartu pertama menentukan bentuk dua putaran berikutnya. Mesin menghitung **rasio keamanan**: dari sisa kartu di tangan Anda, berapa yang bisa disambungkan setelah kartu pertama itu dimainkan.

Nilai	Arti	Tindakan
Rasio 50% atau lebih	aman; banyak pilihan di putaran kedua	boleh diandalkan
Rasio 25-49%	cukup; pilihan terbatas	pertimbangkan alternatif
Rasio di bawah 25%	berbahaya; bisa PASS dini	hindari kecuali terpaksa
Rasio 0%	tidak ada kartu yang bisa dimainkan di putaran kedua	hampir selalu salah

Mesin juga memberi bonus 30 poin rankScore untuk rasio keamanan ini, dan menampilkan daftar kartu mana saja yang bisa dimainkan di putaran kedua — supaya Anda bisa memeriksa perhitungannya dengan mata sendiri.

5.9 Apa yang tidak bisa diberitahu oleh angka ini

Bagian ini penting agar Anda tidak menaruh kepercayaan berlebih.

- **Lawan di simulasi bukan manusia.** Mesin memakai tiga gaya heuristik (penahan, penimbun, pembuang) yang dirotasi. Lawan nyata bisa bermain jauh berbeda, khususnya lawan yang bermain tidak rasional.
- **Pembagian kartu mengasumsikan keacakan.** Setiap simulasi membagikan kartu secara acak di antara lawan, hanya dibatasi oleh PASS dan atribusi Anda. Bila cara mengocok di meja Anda tidak acak (kartu dikocok kurang, kartu menempel), asumsi ini tidak berlaku.
- **Mode cepat memakai derau.** Pada mode biasa (bukan teliti), lawan memilih langkah dengan sedikit keacakan agar variasinya hidup. Mode teliti menghilangkan derau untuk lawan yang bermain tepat setelah Anda.
- **Konvensi meja harus benar.** Bila aturan adu di meja Anda berbeda dari yang disetel, seluruh angka peluang menang dan bar "menang lewat adu" akan menyimpang.
- **Angka tidak pernah menggantikan pengamatan.** Aplikasi ini membutuhkan Anda untuk memasukkan apa yang terjadi; ia tidak melihat meja. Data yang lebih lengkap selalu mengalahkan jumlah simulasi yang lebih besar.

Satu jaminan yang bisa Anda andalkan sepenuhnya: **kejujuran aritmetika**. Hasilnya deterministik (seed tetap), konservasi kartu diverifikasi pada setiap simulasi (penanda `conserved` bernilai benar), dan seluruh angka yang tampil berasal dari perhitungan mesin yang sama yang bisa Anda jalankan sendiri dari terminal.

BAGIAN VI

Arsitektur & Isi Mesin

VI

Bagian ini menjelaskan bagaimana aplikasi dibangun dan mengapa keputusan desainnya diambil. Cocok untuk pembaca yang ingin mengubah kode, memverifikasi perhitungan, atau sekadar tidak ingin memakai alat yang tidak dipahaminya.

6.1 Tiga lapisan



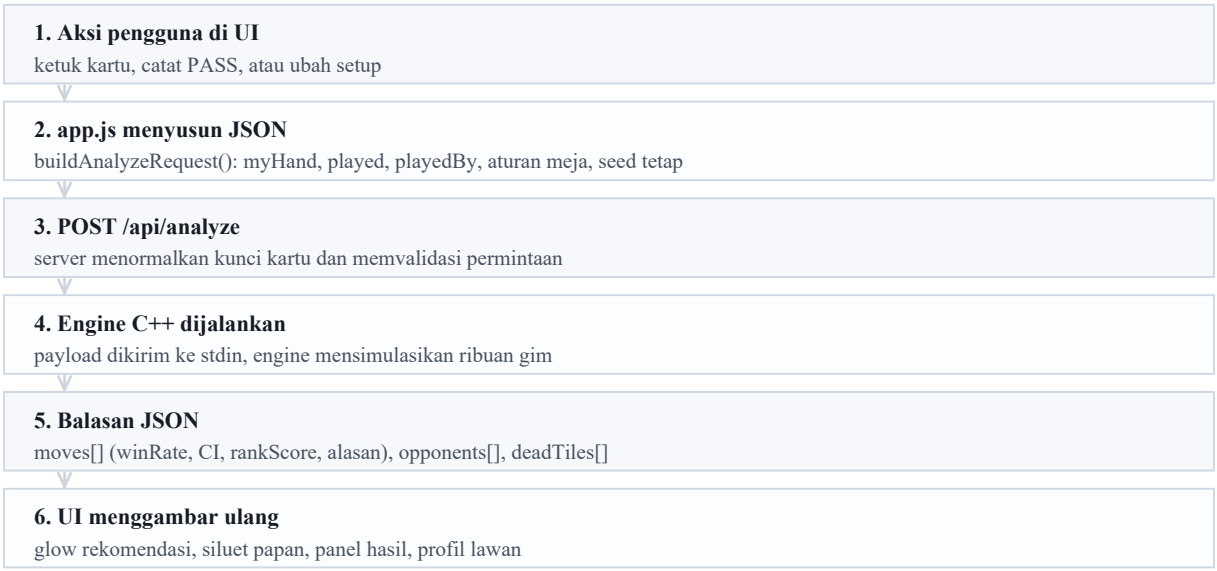
Gambar 5. Tiga lapisan aplikasi: UI web, server Python, dan engine C++ yang dipanggil lewat stdin/stdout berformat JSON.

Pemisahan menjadi tiga lapisan memiliki alasan praktis, bukan sekadar kerapian:

Keputusan	Alasan
Mesin ditulis dalam C++	simulasi Monte-Carlo berjalan milyaran operasi; bahasa terkompilasi memberi kecepatan yang membuat ribuan gim per langkah menjadi layak
Mesin dibuat sebagai program CLI	mesin bisa diuji langsung dari terminal tanpa server, tanpa peramban, dan tanpa jaringan — mempermudah penelusuran dan pengujian otomatis
Server ditulis dengan pustaka bawaan Python	tidak ada dependensi yang perlu dipasang; pemasangan hanya butuh Python
Antarmuka tanpa kerangka kerja	memuat instan di ponsel, bisa dibaca seluruhnya dalam satu berkas app.js, dan tidak butuh proses build

Yang menarik: mesin tidak menghitung apa pun yang berhubungan dengan tampilan. Ia tidak tahu apa itu "glow", "siluet", atau "kolom lawan". Ia menerima JSON, menjalankan simulasi, dan mengembalikan JSON. Semua keputusan visual terjadi di antarmuka.

6.2 Alur satu permintaan



Gambar 6. Alur satu permintaan analisis: dari ketukan di UI sampai hasil tergambar kembali di papan.

Bentuk data yang berpindah antar lapisan:

```
{
  "cmd": "analyze", "numPlayers": 4, "cardsPerPlayer": 7, "numSims": 2000,
  "seed": 20240922, "leftEnd": 3, "rightEnd": 1,
  "myHand": ["6-6", "3-6", "2-3", "1-5", "4-4"],
  "played": ["0-3", "0-2", "1-2"],
  "playedBy": ["opp1", "opp2", "opp1"],
  "nextSeat": 1, "deadlockRule": "lowest", "tieRule": "win",
  "cangkul": true, "adversarial": false,
  "opponents": [{"passes": 1, "eliminated": [3, 1]}, {"passes": 0, "eliminated": []},
                {"passes": 0, "eliminated": []}]
}
```

Balasannya berisi ringkasan permintaan, daftar lawan, daftar langkah berserta seluruh statistiknya, dan (bila ada sisa) daftar kartu yang mungkin mati. Bagian IX memuat tabel lengkap setiap bidang.

Pembagian tanggung jawab — siapa melakukan apa — sengaja dibuat tegas:

Tugas	Dikerjakan oleh	Alasan
Normalisasi format kartu (6-3 → 3-6)	server	masukan pengguna boleh longgar; mesin sebaiknya ketat
Validasi permintaan (batas, duplikat, aturan)	server dan mesin	pertahanan berlapis: server menolak lebih awal, mesin menolak sebagai jaring terakhir
Pembatasan beban CPU (numSims maksimum)	server dan mesin	mencegah penyalahgunaan walau salah satu lapisan diubah
Perhitungan peluang, statistik, dan peringkat	mesin	satu sumber kebenaran; antarmuka hanya menggambar
Penentuan urutan tampilan akhir	antarmuka	hanya urutan tampil; nilai peringkat tetap milik mesin

6.3 Mesin: satu berkas C++20 tanpa dependensi

Mesin berada di `engine/domino_engine.cpp` — sekitar 1.700 baris, satu berkas, tanpa satu pun pustaka pihak ketiga. Ia membaca **satu** permintaan JSON dari masukan standar dan menulis **satu** balasan JSON ke keluaran standar, lalu berhenti.

```
echo '{"cmd":"selftest"}' | ./bin/domino_engine.exe
```

6.3.1 Parser JSON sendiri

Mesin membawa parser JSON kecil (`JValue` + `JParser`). Alasannya sederhana: menambah dependensi JSON hanya untuk membaca satu objek akan mengorbankan sifat paling berharga dari mesin ini — bisa dibangun di mana saja dengan satu perintah.

Parser itu menangani tipe dasar (objek, larik, string, angka, boolean, null), escape `\uXXXX` dengan pengkodean UTF-8, dan **menolak** masukan yang tidak rapi (karakter sisa setelah nilai JSON, string tanpa penutup, angka tidak sah). String sumber disimpan sebagai nilai, bukan rujukan, supaya tidak ada rujukan menggantung saat data berasal dari sementara.

6.3.2 Kartu dan kunci kanonik

Kumpulan kartu dibuat sekali di awal: `0-0`, `0-1`, … `6-6`, 28 buah, dengan kunci berbentuk `lo-hi` (angka kecil lebih dulu). Karena itu mesin **hanya menerima** kunci kanonik:

```
diterima mesin   : "3-6", "0-0", "1-5"
ditolak mesin    : "6-3"   -> invalid tile key: 6-3
```

Normalisasi `6-3` menjadi `3-6` adalah tugas server. Ini contoh nyata pembagian tanggung jawab di atas: pengguna boleh mengetik apa saja, mesin tetap ketat.

6.3.3 Angka acak yang bisa diulang

Mesin memakai pembangkit acak **xoshiro256**** yang disemai lewat `splitmix64`. Sifat yang penting: dengan seed yang sama, urutan angka yang dihasilkan identik, sehingga seluruh hasil analisis dapat direproduksi persis. Antarmuka selalu mengirim seed tetap `20240922`, jadi sesi yang sama akan menghasilkan angka yang sama pula — kecuali jumlah simulasi diubah.

Ini bukan kemewahan teknis. Determinisme membuat hasil bisa diperiksa: bila ada yang mencurigakan, jalankan permintaan yang sama dua kali di terminal dan bandingkan keluarannya huruf demi huruf.

6.3.4 Peta angka dan dominasi

Dua struktur data kecil dipakai berulang kali: **Dominance** (berapa kali setiap angka 0-6 muncul di sebuah tangan, dengan ambang "dominan" pada 3 dan "kuat" pada 2) dan **NumMap** (berapa kartu setiap angka ada di tangan saya, sudah keluar, dan belum diketahui). NumMap selalu memenuhi `mil ik saya + sudah keluar + belum diketahui = 7`, yang sekaligus menjadi pemeriksaan konsistensi.

6.3.5 Pembagian kartu yang sadar batasan

Ini bagian yang membuat "data PASS" dan "atribusi kartu" benar-benar berguna. Setiap simulasi harus membagikan kartu yang belum diketahui ke tangan lawan-lawannya. Aturannya bertingkat:



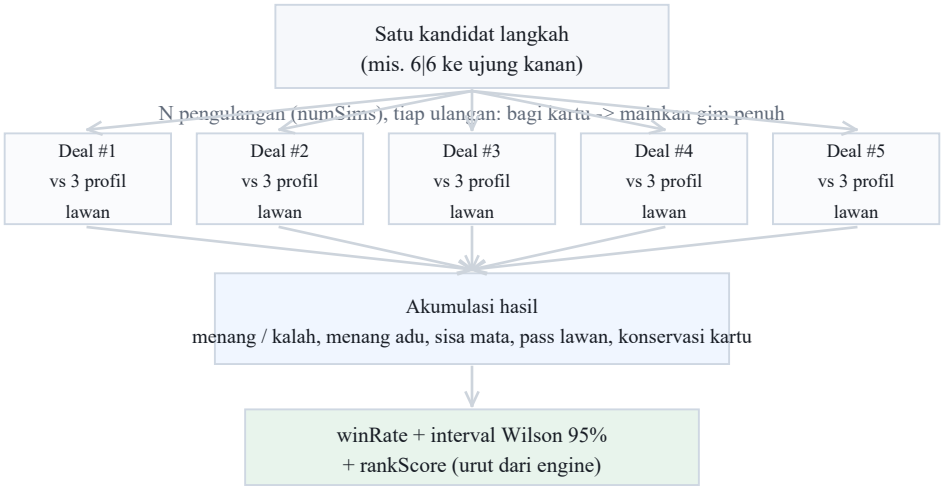
Gambar 7. Pembagian kartu secara sadar-batasan (constraint-aware dealing): bukti kuat diprioritaskan, batasan dilepas hanya bila kuota mustahil dipenuhi.

Prioritas	Sumber data	Perlakuan
0 (terkuat)	atribusi kartu Anda	kartu dipaksa masuk ke tangan lawan pemiliknya
1	PASS lawan	kartu yang mengandung angka yang di-PASS dibuang dari undian untuk lawan itu
2 (cadangan)	—	bila kuota lawan belum terpenuhi, batasan dilepas dan kejadian itu dihitung sebagai fallback

Lawan dengan batasan paling ketat dilayani lebih dulu, supaya kartu yang paling terbatas tidak "terpakai" oleh pembagian yang lebih longgar. Jumlah *fallback* dilaporkan oleh pemeriksaan mandiri: pada 400 skenario acak, sekitar 57 skenario terpaksa memakai jalur cadangan — angka yang wajar, bukan tanda kerusakan.

Perhatikan konsekuensi pentingnya: bila atribusi Anda bertentangan dengan PASS lawan yang sama (misalnya lawan itu sudah ditandai tidak punya angka 3, tetapi Anda menandainya sebagai pemain kartu `3-5`), mesin **mengabaikan atribusi yang bertentangan** dan memakai bukti PASS. Bukti keras menang atas dugaan.

6.4 Mesin: satu langkah dianalisis bagaimana



Gambar 8. Monte-Carlo per kandidat langkah: setiap ulangan memakai pembagian kartu dan gaya lawan yang berbeda, lalu hasilnya diagregasi.

Untuk **setiap** kandidat langkah, mesin melakukan rangkaian berikut:

1. **Hasilkan langkah yang sah.** Sebuah kartu menghasilkan langkah "ke kiri", "ke kanan", atau keduanya; pada papan kosong hanya satu langkah "pertama". Kartu yang bisa masuk ke dua ujung dihitung sebagai dua kandidat terpisah, karena akibatnya berbeda.
1. **Terapkan langkah secara maya** untuk mendapatkan ujung baru dan sisa tangan, lalu susun peta angka baru untuk keadaan setelah langkah itu.
1. **Hitung heuristik tanpa simulasi** (di bawah).
2. **Jalankan N simulasi.** Setiap simulasi: bagi kartu dengan menghormati batasan → mainkan gim sampai selesai → catat hasilnya (menang/kalah, sisa mata, jumlah PASS lawan, konservasi kartu, kartu yang dimainkan lawan pada gim yang kalah).
1. **Agregasi**: peluang menang, selang Wilson, peluang menang lewat adu, rata-rata sisa mata, rata-rata PASS lawan, skor risiko.
1. **Susun rankScore** dan kirimkan seluruh statistiknya ke antarmuka.

6.4.1 Heuristik tanpa simulasi

Empat penilaian dihitung hanya dari keadaan papan dan peta angka:

Penilaian	Pemicu	Poin
Dukungan dominasi	ujung cocok dengan angka dominan Anda	+20
Dukungan dominasi	kedua ujung adalah angka dominan Anda	+30
Dukungan angka kuat	ujung cocok dengan angka yang Anda pegang dua kali	+8
Jerat	angka dominan Anda terpasang di ujung dan masih ada kartu di luar	+15
Jerat ganda	kedua ujung adalah angka dominan yang sama	+20
Kunci	ujung dengan angka yang tidak ada di luar, dan Anda masih memegangnya	+25
Kunci hampir	angka ujung tinggal satu di luar dan Anda pegang dua	+12
Jebakan diri	ujung mati untuk semua orang dan bukan milik Anda	+10 (pengurang)
Jebakan diri	Anda tidak punya kartu berangka ujung sementara masih ada di luar	+6 (pengurang)
Blok terjamin	lawan yang pasti tidak bisa menyambung menurut data PASS	+8 per lawan

6.4.2 Tiga gaya lawan

Simulasi tidak bermain melawan satu jenis lawan. Tiga profil disimulasikan dan **dirotasi** antar simulasi, sehingga tiap kandidat langkah diuji menghadapi campuran gaya:

Gaya	Perilaku (bobot penilaiannya)
Blocker	menghukum ujung yang masih banyak jawabannya, menghargai keluwesan dan nilai mata; sangat menghindari membuat kedua ujung sama

Hoarder	menghargai ujung dari angka kuatnya dan menghukum pemborosan kartu angka dominan (kartu dominan bernilai -25, balak dominan -40)
Dumper	cepat membuang: nilai mata besar dan balak diberi bobot positif besar
Campuran (pemain)	gaya yang Anda pakai sendiri di simulasi: menghargai balak, ujung berangka dominan, ujung yang hanya bisa dijawab Anda, dan keluwesan

Pada mode biasa, setiap pilihan diberi derau kecil (nilai acak sampai 5) agar variasinya hidup dan tidak menghasilkan permainan yang sama terus. Pada **mode teliti**, lawan yang bermain tepat setelah Anda dimainkan sebagai **Blocker tanpa derau** — permainan yang ketat dan deterministik. Efeknya: estimasi menjadi lebih konservatif, seperti bermain melawan lawan yang tidak pernah melakukan kesalahan bodoh.

6.5 Mesin: satu gim simulasi

Satu gim berjalan seperti permainan sungguhan:

- Giliran dimulai dari pemain yang bermain setelah Anda (atau lawan pertama bila papan kosong), lalu berputar.
- Pemain yang tidak punya langkah sah **PASS**. Bila mode cangkul aktif dan masih ada sisa, ia menarik kartu sisa **satu per satu** sampai menemukan kartu yang bisa dimainkan atau sisa habis.
- Gim berakhir pada salah satu dari tiga kemungkinan: tangan Anda habis (**menang**), tangan lawan habis (**kalah**), atau semua pemain PASS dalam satu putaran penuh (**macet** → dinilai dari adu).
- Ada batas keselamatan 300 giliran; bila tercapai, gim dinilai seperti keadaan macet.

Batas ini praktis tidak pernah tercapai, tetapi menjamin simulasi selalu berhenti.

Setiap gim memverifikasi **konservasi kartu**: jumlah kartu di semua tangan ditambah kartu yang sudah masuk papan harus sama dengan jumlah kartu saat pembagian. Bila tidak sama, hasil itu ditandai `conserved: false` dan pemeriksaan mandiri akan menangkapnya. Ini pertahanan terhadap kelas kesalahan yang paling berbahaya dalam simulasi: kartu yang "muncul dari udara" akan menghasilkan peluang menang yang salah tanpa gejala apa pun.

Penentuan pemenang saat macet memakai konvensi meja Anda: nilai mata Anda dibandingkan dengan nilai mata **terkecil** lawan atau **rata-rata** lawan, dengan perilaku seri sesuai pilihan Anda.

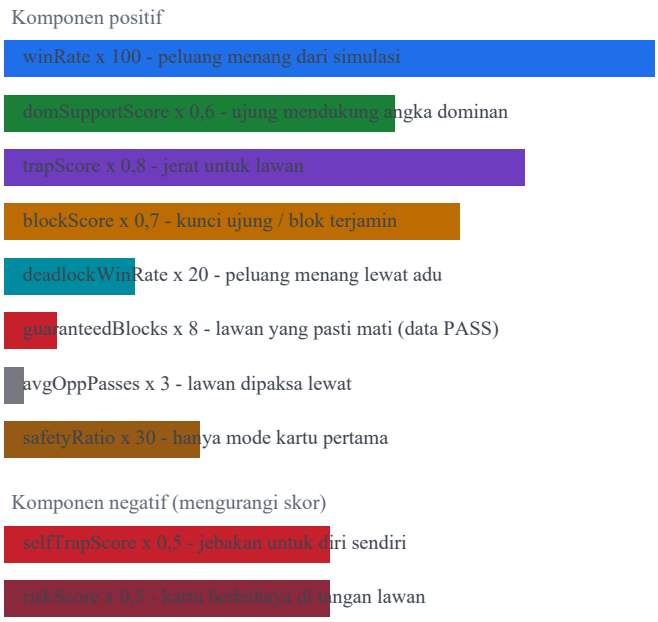
6.6 Mesin: dari hasil ke rekomendasi

Setelah semua simulasi untuk semua kandidat selesai, mesin menghitung tiga hal lagi.

Skor risiko. Untuk setiap gim yang Anda kalahi, mesin mencatat kartu-kartu yang dimainkan lawan. Lima kartu yang paling sering muncul dihitung frekuensinya terhadap jumlah kekalahan; jumlah kontribusinya (frekuensi x 10) menjadi `riskScore`. Inilah dasar kotak "Kartu Berbahaya" di antarmuka.

Keamanan kartu pertama. Hanya untuk papan kosong: dari sisa kartu di tangan Anda, berapa yang bisa dimainkan setelah kandidat pertama dipakai.

Peringkat. Semua komponen digabung menjadi satu nilai:



Gambar 9. Susunan rankScore di engine. Semua bobot dihitung di C++ supaya urutan kartu tidak berbeda antara tampilan dan mesin.

Contoh perbandingan nyata dari Bagian V: kartu `2-3` memiliki peluang menang 71,2% (lebih tinggi 2 poin daripada `3-6`), tetapi kalah peringkat karena `3-6` memasang jerat (+15 x 0,8 = +12) dan tidak memiliki jebakan diri, sementara `2-3` memiliki jebakan diri (-6 x 0,5 = -3) dan tidak memasang jerat. Selisih akhir: 84,2 berbanding 60,4.

Kartu yang mungkin mati. Bila masih ada sisa kartu, mesin menjalankan simulasi pembagian tambahan (tanpa memainkan gim) untuk menghitung peluang setiap kartu berakhir di sisa, serta perkiraan jumlah kartu mati per angka. Jumlah simulasi tambahan ini dibatasi 2.000 agar tidak menambah waktu tunggu secara berlebihan.

6.7 Server: tipis, tetapi tidak bodoh

Server sengaja tipis: ia menormalkan, memvalidasi, memanggil mesin, dan menyajikan berkas. Dua hal yang membuatnya tidak sekadar "pipa":

Pembatalan yang benar-benar membatalkan. Peramban yang menekan tombol Batal menutup koneksinya, dan server memantau hal itu setiap 0,2 detik (`select` + `MSG_PEEK`). Begitu koneksi hilang, proses mesin dimatikan — penting pada tier 1,7 juta simulasi, karena tanpa ini CPU tetap bekerja penuh (dan antrean ikut terhambat) hanya untuk hasil yang tidak akan pernah ditampilkan.

Antrean mesin. Satu proses mesin memakai seluruh inti CPU (`DOMINO_THREADS`). Bila beberapa permintaan datang bersamaan tanpa pengaturan, mereka saling merebut inti yang sama sehingga setiap analisis menjadi lebih lambat dan waktu tunggunya tak terduga. Karena itu server menjalankan mesin bergiliran (bawaan `DOMINO_MAX_ENGINE=1`), dengan antrean berbatas `DOMINO_ENGINE_WAIT` detik sebelum menjawab 503. Satu permintaan pada satu waktu berarti setiap analisis mendapat kecepatan penuh.

Kode status HTTP yang jujur. Kesalahan tidak pernah menyamar sebagai keberhasilan.

Keadaan	Status	Alasan
berhasil	200	—

validasi permintaan gagal	400	kesalahan pengirim, bukan server
mesin menolak permintaan (logika)	400	ok:false dari mesin tidak boleh jadi 200
token salah atau hilang	401	hanya saat APP_TOKEN diisi
endpoint tidak dikenal	404	—
metode salah (misalnya GET ke /api/meta)	405	—
mesin gagal dijalankan (exit != 0)	500	detail hanya masuk log server
balasan mesin bukan JSON	502	mesin berjalan tetapi keluarannya rusak
biner mesin belum ada	503	pemasangan belum lengkap
antrean mesin penuh (menunggu > DOMINO_ENGINE_WAIT)	503	semua slot mesin sibuk
mesin melebihi 300 detik	504	permintaan terlalu berat

Prinsip desainnya: klien luar (skrip, monitor, `curl`) harus bisa menyimpulkan berhasil atau gagal **han ya dari kode status**, tanpa perlu membaca isi balasan.

Keamanan bawaan. Karena aplikasi ini sering dijalankan di komputer pribadi dan kadang dibuka lewat tunnel, servernya memasang pengaman dasar:

Pengaman	Cara kerja
Mencegah sniffing tipe berkas	header X-Content-Type-Options: nosniff
Mencegah pembingkaian (clickjacking)	Content-Security-Policy dengan frame-ancestors 'none'
Membatasi sumber skrip	default-src 'self'; script-src 'self' — tidak ada skrip pihak ketiga
Mencegah kebocoran rujukan	Referrer-Policy: no-referrer
Mencegah pembacaan berkas di luar web/	setiap jalur di-resolve dan diperiksa berada di dalam web/
Mencegah CSRF dari situs lain	isi permintaan hanya diterima bila Content-Type: application/json, dan Origin (bila ada) harus sama dengan Host
Membatasi ukuran permintaan	maksimum 1 MiB; badan kosong atau lebih besar ditolak
Mencegah pesan galat memantulkan masukan	pesan galat menyebutkan posisi elemen, bukan isi yang dikirim
Token pembanding waktu-konstan	hmac.compare_digest — membandingkan token dengan waktu yang selalu sama

Dua pengaman terakhir layak dijelaskan lebih dalam. Yang pertama menutup celah XSS: pesan galat dikirim ke antarmuka dan pernah dirender sebagai HTML, jadi masukan pengguna tidak boleh dipantulkan mentah-mentah. Yang kedua menutup celah CSRF yang khas untuk aplikasi lokal: peramban mengirim permintaan "sederhana" lintas situs tanpa pemeriksaan awal, jadi halaman web lain di komputer Anda bisa saja memicu `POST /api/analyze` ke localhost. Pemeriksaan `Content-Type` dan `Origin` menutup jalan itu.

6.8 Antarmuka: satu berkas, satu sumber keadaan

`web/app.js` memuat seluruh logika antarmuka dalam satu objek keadaan:

Bidang keadaan	Isi
myHand	daftar kartu di tangan Anda, dalam bentuk kunci kanonik
boardTiles	daftar kartu di papan, berisi kartu yang sudah diorientasikan, pemilik, dan penanda kartu pertama
allPlayed	daftar kronologis semua kartu yang pernah masuk papan
playedBy	label pemilik untuk setiap kartu di allPlayed (me, unknown, opp1…opp4)
opponents	untuk setiap lawan: riwayat PASS dan daftar angka yang tereliminasi
pending	kartu yang sedang menunggu pilihan ujung
pendingAttrib	kartu yang sedang menunggu atribusi pemain
isAnalyzing, pendingRun, abortCtl	penanda analisis berjalan, antrean, dan pembatal
liveMode	apakah mode hidup aktif
lastAnalysis, recommended	hasil terakhir dan rekomendasi #1
timeline	salinan keadaan untuk Undo (maksimum 100 langkah)

Dua bidang yang perlu diperhatikan: `boardTiles` **bukan** daftar kronologis (kartu yang dimainkan ke kiri disisipkan di depan larik), sehingga penanda `first: true` dipakai khusus untuk menandai kartu pertama; sedangkan `allPlayed` tetap kronologis dan dipakai untuk menghitung kartu yang sudah keluar. Perbedaan ini adalah sumber kebingungan yang sering muncul saat memodifikasi kode, dan sudah diatasi dengan penanda eksplisit.

Keamanan tampilan. Semua nilai dinamis yang masuk ke HTML dilewatkan fungsi pengaman `esc()` yang mengubah karakter `&`, `<`, `>`, `"`, dan `'` menjadi bentuk aman. Ini penting karena data sesi bisa berasal dari berkas impor.

Validasi berkas impor. Sebelum diterima, sesi yang diimpor diperiksa berlapis: setiap kartu harus cocok dengan pola `angka-angka`, label pemain harus sah, dan data PASS harus berupa angka 0-6. Data yang tidak lolos ditolak seluruhnya, bukan sebagian.

Ketahanan Undo. Setiap aksi yang mengubah keadaan menyimpan salinan lengkap (termasuk PASS lawan dan atribusi), sehingga Undo tidak pernah meninggalkan data setengah jalan.

6.9 Tata letak papan: mengapa ada "pengecilan otomatis"

Menampilkan rantai domino hingga puluhan kartu di layar ponsel adalah masalah nyata. Pendekatan naif (satu baris panjang yang digulir) membuat pemain kehilangan gambaran besar. Karena itu papan memakai tiga teknik bertingkat:

1. **Baris ular (boustrophedon).** Kartu disusun dalam baris-baris yang bergantian arah.

Baris ganjil dibalik urutannya **dan** tiap kartunya dicerminkan, sehingga angka yang bersambung tetap bertemu rapat di tikungan. Tikungan dibentuk oleh kartu "siku" yang digambar berdiri.

1. **Pengecilan bertahap.** Ukuran kartu diturunkan sampai seluruh isi papan muat (batas bawah setengah ukuran normal). Skala ini diterapkan lewat satu variabel CSS (`--tw`), dan semua ukuran turunan (kartu berdiri, pips, garis pemisah) ikut mengecil secara proporsional.
1. **Pemusatan kartu pertama.** Kartu pertama diusahakan selalu berada di tengah area papan. Bila isi papan muat, seluruh baris digeser (tanpa menggulir). Bila isi papan lebih besar dari papan bahkan setelah pengecilan, aplikasi kembali ke mode gulir dan memakai tambahan lebar/padding untuk mencapai posisi tengah.

Ada satu detail yang tampak sepele tetapi krusial: aplikasi mencoba dua gaya pembagian baris — **baris penuh** (kartu paling besar, tetapi kartu pertama belum tentu di tengah) dan **baris lebih pendek sekitar 60%** (kartu sedikit lebih kecil, tetapi kartu pertama bisa tepat di tengah). Pemenangnya dipilih dengan urutan prioritas: muat dulu, baru paling dekat ke tengah, baru ukuran kartu paling besar.

Pemantauan ukuran. Lebar kolom papan bisa berubah tanpa perubahan ukuran jendela (misalnya saat panel hasil selesai digambar dan mengubah tata letak). Karena itu papan memantau kontainernya sendiri lewat `ResizeObserver`, dengan penyaring penting: perubahan lebar kecil (di bawah 4 piksel) diabaikan bila berasal dari muncul-hilangnya bilah gulir papan itu sendiri — tanpa penyaring itu, tata letak bisa berbalas-balasan selamanya.

6.10 Siluet rekomendasi (ghost)

Siluet kartu rekomendasi digambar sebagai **lapisan di atas** papan, bukan sebagai kartu di dalam baris. Alasannya: menambahkan kartu maya ke dalam baris akan mengubah panjang baris dan bisa menggeser susunan kartu asli — efek yang menyesatkan. Sebagai lapisan, ia diposisikan relatif terhadap kotak kartu pertama:

Arah rekomendasi	Posisi siluet	Orientasi
Ke kanan	di atas kepala kartu pertama	berdiri; separuh yang cocok di atas
Ke kiri	di kaki kartu pertama	berdiri; separuh yang cocok di bawah
Papan kosong	di dalam alur baris	berdiri (dia calon kartu pertama)

Label (misalnya "TARUH DI KANAN") ditulis di samping siluet, karena bila diletakkan di atasnya label itu keluar dari area papan dan tidak terbaca. Siluet digambar **setelah** pengecilan dan pemusatan selesai, supaya koordinatnya mengikuti kotak kartu pertama yang sudah final.

6.11 Mode hidup, antrean, dan pembatalan

Tiga mekanisme kecil yang membuat mode hidup terasa mulus:

- **Antrean (`pendingRun`).** Bila analisis sedang berjalan dan Anda melakukan aksi lain, permintaan baru tidak dijalankan bersamaan (bisa menghabiskan CPU), tetapi ditandai. Saat analisis berjalan selesai, permintaan terbaru dijalankan sekali lagi.
- **Pembatalan (`AbortController`).** Menekan tombol saat analisis berjalan membatalkan permintaan yang sedang berjalan, sehingga komputer tidak terus bekerja untuk hasil yang sudah tidak Anda butuhkan.
- **Penjagaan keadaan.** Undo dan penekanan kartu dinonaktifkan saat analisis berjalan,

supaya keadaan tidak berubah di tengah perhitungan dan hasil yang tampil selalu cocok dengan data yang ada di layar.

6.12 Service worker: dua strategi berbeda

Jenis permintaan	Strategi	Alasan
Berkas antarmuka (HTML, CSS, JS, ikon)	ambil dari tembolok, perbarui di latar belakang	aplikasi terbuka instan dan tetap bisa dibuka tanpa koneksi
Permintaan /api/*	selalu ke jaringan	hasil analisis tidak boleh diambil dari tembolok

Karena berkas statis diambil dari tembolok lebih dulu, versi terbaru hanya terlihat setelah nama tembolok di `sw.js` diubah. Ini konsekuensi yang disengaja dari strategi tersebut, dan bagian pemecahan masalah akan mengingatkan Anda bila perubahan kode tidak kunjung tampil.

6.13 Riwayat versi

Versi	Perubahan besar
v1	analisis dasar satu lapisan (sebelum pemisahan mesin)
v2	pemisahan mesin C++ + server Python; simulasi, peta angka, konvensi meja
v3	atribusi kartu per lawan, tiga profil lawan, mode teliti, Wilson, rankScore, PWA
v3.1	mode cangkul & kartu sisa mati, kisi kandidat kartu per lawan, masukan angka
v3.2	mode hidup aktif sejak awal, masukan cepat dua kotak, indikator status di judul
v3.3	siluet rekomendasi (ghost), glow biru pada kartu pilihan, perbaikan arah ujung lawan
v3.6	tata letak papan bergaya ular dengan pengecilan otomatis dan pemusatan kartu pertama
perbaikan terakhir	rumus interval Wilson diperbaiki (sebelumnya selalu menghasilkan 0-100%)

Nomor versi mesin berdiri sendiri: mesin melaporkan dirinya sebagai `domino-cpp/3.0.0` pada setiap balasan, sehingga Anda selalu bisa memastikan versi mesin yang sedang dipakai.

BAGIAN VII

Pengujian & Kualitas

VII

Aplikasi ini adalah alat pengambil keputusan. Kesalahan di dalamnya tidak muncul sebagai kerusakan yang terlihat, melainkan sebagai **angka yang salah dengan tampilan yang meyakinkan**. *Karena itu pengujiannya tidak berhenti pada "programnya jalan", melainkan memeriksa sifat-sifat yang harus selalu benar — invarian.*

7.1 Sebelas pemeriksaan mandiri mesin

Mesin membawa pemeriksaannya sendiri. Jalankan:

```
python tools/selftest.py
```

Mesin menerima perintah `selftest` dan mengembalikan daftar pemeriksaan beserta hasilnya.

#	Nama pemeriksaan	Yang dijamin
1	deck_28_kartu	tepat 28 kartu unik dari 0-0 sampai 6-6
2	parser_json	parser bawaan membaca objek, larik, angka, boolean, dan null dengan benar
3	determinisme_seed	analisis yang sama dengan seed sama menghasilkan keluaran identik
4	deal_konservasi_&_constraint	pada 400 skenario acak: kartu tidak hilang/berganda, batasan PASS dipatuhi, dan kartu hasil atribusi benar-benar masuk ke tangan yang tepat
5	simulasi_invariant	500 gim acak selalu berhenti, konservasi kartu terjaga, dan statistik wajar
6	boneyard_konservasi	pada 2 pemain, sisa kartu selalu 14 dan semua kartu tetap terhitung
7	ranking_rankScore	daftar langkah dari mesin benar-benar terurut menurun menurut rankScore
8	aturan_adu	peluang menang dengan "seri = kalah" tidak pernah melebihi "seri = menang"

9	cangkul_mode	bila cangkul dimatikan, tidak ada satu pun sambaran; dengan cangkul, konservasi tetap terjaga
10	atribusi_endgame	permainan akhir dengan 12 kartu lawan yang diatribusikan tetap bisa dianalisis (regresi dari kesalahan lama)
11	interval_wilson	interval keyakinan mengurung peluang terukur, nilainya benar untuk kasus uji, dan menyempit saat simulasi ditambah

Keluaran nyata dari mesin versi ini:

```
[selftest] [PASS] deck_28_kartu - 28 tile unik 0-0 .. 6-6
[selftest] [PASS] parser_json - ok
[selftest] [PASS] determinisme_seed - 2 pemain, seed tetap, 2x analisis identik
[selftest] [PASS] deal_konservasi_&_constraint - 400 skenario acak | fallback terjadi di 57 skenari
o (informasi)
[selftest] [PASS] simulasi_invariant - 500 gim acak berhenti & statistik konsisten
[selftest] [PASS] boneyard_konservasi - 200 gim 2-pemain, semua tile tetap terhitung 28
[selftest] [PASS] ranking_rankScore - moves terurut rankScore desc
[selftest] [PASS] aturan_adu - winRate(lose) <= winRate(win) pada semua langkah
[selftest] [PASS] cangkul_mode - cangkul=false -> 0 nyamber; cangkul=true -> konservasi
[selftest] [PASS] atribusi_endgame - end-game: 12 kartu teratribusi lawan, analisis tetap jalan
[selftest] [PASS] interval_wilson - w=346/500 -> 0.650..0.731, lebar menyempit saat n naik
[selftest] semua check LOLOS
```

Perhatikan baris nomor 4: laporan itu menyebutkan bahwa pada 57 dari 400 skenario, jalur cadangan terpakai. Itu **bukan kegagalan** — pemeriksaan tetap lulus karena konservasi dan kepatuhan batasan diverifikasi hanya pada skenario yang tidak memakai jalur cadangan, dan jumlah fallback dilaporkan sebagai informasi. Prinsipnya: bila batasan yang Anda berikan secara matematis mustahil dipenuhi (misalnya kuota kartu lebih besar daripada kartu yang tersedia setelah eliminasi), mesin harus tetap berjalan dan tetap jujur tentang itu.

7.2 Dua puluh sembilan uji server

```
python tools/test_server.py
```

Suite ini menjalankan server sungguhan pada port sementara dan menguji tiga kelompok.

Normalisasi format kartu (4 tes). Bentuk kanonik (`3-5`, `6-1`), pemisah ramah (`6|1`, `3:2`, `4 5`, `61`), bentuk tidak sah yang harus ditolak (`7-1`, `a-b`, kosong, `1-2-3`, angka bukan string), dan satu tes khusus yang memastikan **pesan galat tidak pernah memantulkan masukan mentah** — termasuk ketika masukan itu berisi skrip HTML.

Validasi permintaan (8 tes). Permintaan yang sah diterima; `nextSeat` di luar batas ditolak; aturan adu dan aturan seri yang tidak dikenal ditolak; panjang `playedBy` dan label pemain diperiksa; kartu yang muncul di tangan dan papan sekaligus ditolak; serta batas `numSims` (1 sampai 1700000, nilai aneh seperti `10^12`, bilangan negatif, teks, dan nilai kosong semuanya ditolak) — pengaman anti pembebanan CPU.

Dua tes terakhir kelompok ini menguji **pembatalan**, karena pada tier 1.700.000 tombol Batal bukan sekadar kenyamanan: tanpa itu seluruh inti CPU tetap terpakai puluhan detik.

Tes	Yang dibuktikan

client_gone_detects_disconnect	client_gone benar-benar membedakan koneksi yang masih hidup dari yang sudah ditutup klien, dan tidak memakan data yang masih antre
run_engine_cancel_kills_process	proses mesin yang sedang menghitung 400.000 simulasi berhenti < 5 detik setelah pembatalan diminta, dan tidak menghasilkan JSON utuh

Uji langsung terhadap server (17 tes). Endpoint kesehatan, `/api/meta` yang menormalkan masukan dan hanya mengembalikan kartu yang benar-benar ada, analisis end-to-end, mode cangkul mati beserta kartu sisa mati, atribusi `playedBy`, kasus permainan akhir dengan banyak atribusi, dan kontrak keamanan:

Tes keamanan	Yang dibuktikan
security_headers_present	header nosniff, no-referrer, dan CSP ada di berkas statis maupun balasan API
reject_non_json_content_type	text/plain dan application/x-www-form-urlencoded ditolak (vektor CSRF peramban)
reject_cross_origin	Origin yang berbeda dari Host ditolak, sementara Origin yang sama diterima
numSims_out_of_range_rejected	permintaan berat ditolak dengan status 400
malformed_bodies_clean_400	badan non-objek dan JSON bersarang 5000 tingkat dijawab 400 dengan rapi, bukan memutus koneksi
error_over_http_never_echoes_input	masukan berisi skrip tidak muncul di pesan galat
error_never_returns_http_200	lima kasus galat, termasuk yang lolos validasi server tetapi ditolak mesin, semuanya bukan 200
ok_false_body_always_paired_with_non_200	aturan umum: setiap balasan ok:false selalu disertai status 4xx atau 5xx
unknown_endpoint_and_method_still_4xx	endpoint tidak dikenal menjawab 404
meta_only_real_tiles	/api/meta tidak lagi mengembalikan kartu mustahil seperti 0-7 (regresi lama)

Dua tes terakhir pada kelompok pertama dan beberapa tes di atas adalah contoh tes yang **lahir dari kesalahan nyata**: setiap kali sebuah cacat ditemukan, tesnya ditambahkan supaya cacat itu tidak bisa kembali tanpa ketahuan.

Hasil menjalankan suite di mesin ini:

```
Ran 29 tests in 1.093s
OK
```

7.3 Paralelisasi: output harus tidak berubah (4 tes)

Mesin menjalankan simulasi Monte-Carlo dengan beberapa thread sekaligus (`DOMINO_THREADS`, bawaan semua inti logis). Satu hal yang tidak boleh terjadi adalah jumlah thread mengubah hasil: pembagian blok kerja memakai snapshot stream bilangan acak yang sama, sehingga urutan undian tiap simulasi tetap identik.

Jalankan `python tools/test_engine_parallel.py`:

Tes	Yang dibuktikan
parallel_equals_sequential	output 1 thread sama persis dengan 2 dan 8 thread pada empat skenario (4 pemain, adversarial, 2 pemain, batas 2000 sim)
default_threads_matches_sequential	mode bawaan (semua inti) sama dengan mode sekuensial
threads_zero_or_negative_means_sequential	DOMINO_THREADS=0 dan angka negatif tidak membuat mesin gagal, dan tetap sekuensial
empty_unknown_pool_does_not_crash	permintaan dengan pool kartu tak diketahui kosong (2 pemain x 14 kartu) tidak lagi mematikan proses

Ditambah suite beban penuh (dijalankan dengan `DOMINO_SLOW_TESTS=1`, ≈1 menit) untuk tier 1.700.000 simulasi — beban maksimum yang ditawarkan aplikasi:

Tes	Yang dibuktikan
test_no_crash_and_ok	mesin tidak jatuh dan tidak memangkas numSims pada beban penuh
test_counts_and_conservation_at_full_load	wins + losses == numSims, rincian kemenangan konsisten, konservasi kartu terjaga, dan lebar interval < 0,2 poin (janji ±0,1%)
test_rankScore_sorted	urutan peringkat tetap benar pada beban penuh
test_full_load_parallel_equals_sequential	output 1 thread, 4 thread, dan 12 thread sama persis pada 1,7 juta simulasi

Hanya dua jalur pembatalan yang diuji di suite ini (level soket): pembatalan yang benar saat pengguna menekan Batal juga sudah diverifikasi dari sisi antarmuka (lihat akhir bab ini). Bukti nyata dari jalur browser: log server mencatat `[engine] dibatalkan: klien menutup koneksi`, dan tidak ada satu pun proses `domino_engine` yang tertinggal.

Dua tes terakhir adalah pengaman regresi untuk cacat nyata: `shuffle` pada vector kosong dulu memakai indeks `SIZE_MAX` lalu `nextInt(0)` (modulo nol) — proses mati dengan `0xC0000094`/`0xC0000005`.

7.4 Verifikasi identik terhadap build sebelumnya

Perubahan pada jalur perhitungan tidak boleh dinilai hanya dari "masih jalan". `tools/audit_golden.py` membandingkan keluaran mesin **byte-per-byte**:

```
python tools/build_baseline.py                    # bangun mesin dari git HEAD + rekam golden
python tools/audit_golden.py verify "$TEMP/domino_audit_head"
```

Sembilan skenario diuji: 2/3/4 pemain, mode adversarial, cangkul mati, kartu pertama, atribusi `playedBy`, 5000 simulasi (jalur paralel), batas 2000 simulasi, dan pool kartu kosong. Semuanya harus melaporkan `IDENTIK`. Skrip mencatat juga bila mesin lama **crash** pada suatu skenario, lalu memeriksa bahwa crash itu sudah hilang.

7.5 Mengukur kecepatan

```
python tools/bench_engine.py     # mesin HEAD vs optimasi 1 thread vs optimasi paralel
```

Contoh keluaran di mesin uji pada beban 50.000 simulasi x jumlah kandidat langkah (AMD Ryzen 5 5625U — **6 inti fisik / 12 utas logis**; "12 thread" di bawah berarti 12 utas):

Versi	Waktu	Laju	Relatif
build HEAD	2,62 s	57.344 sim/detik	1,00x
optimasi 1 thread	1,03 s	146.042 sim/detik	2,55x
optimasi paralel (12 thread)	0,20 s	770.762 sim/detik	13,44x

7.5.1 Beban penuh (tier 1.700.000) yang terukur

Tier tertinggi bukan sekadar angka di dokumen; ia diukur, dan invariannya diperiksa pada skala itu. Semua angka di bawah berasal dari mesin uji di atas, 4 pemain, cangkul aktif, 7 kandidat langkah (papan kosong, 7 kartu di tangan):

Yang diukur	Hasil	Arti
Waktu satu analisis penuh	22,0 s (11,9 juta simulasi-langkah)	jauh di bawah batas 300 detik, jadi tidak ada hasil yang terpotong
Laju	602.081 simulasi-langkah/detik (12 utas)	"1,7 juta simulasi" per kartu selesai dalam ~3 detik
Efisiensi paralel	5,23x pada 12 utas = 87,2% dari 6 inti fisik (6 utas memberi 3,79x)	sisa ~13% adalah bagian yang memang sekuensial (perakitan hasil, simulasi sisa kartu)
Hasil 1 vs 6 vs 12 utas	identik byte-per-byte	jumlah utas tidak pernah mengubah angka
Dua run beban penuh	identik byte-per-byte	deterministik sampai skala maksimum
Setengah-lebar interval terlebar	0,0752 poin (janji: ≤ 0,1)	klaim ±0,1% pada tier ini terbukti, bukan perkiraan
Konservasi	wins + losses == 1.700.000 dan conserved: true pada semua 7 langkah	tidak ada kartu atau gim yang hilang karena bebannya besar

Beban besar juga diuji dari sisi yang paling mudah dilupakan: **apakah aplikasinya masih enak dipakai selagi mesin bekerja?** Selama run 22 detik yang memakai seluruh 12 utas, antarmuka diukur langsung: **59 frame/detik** (frame terburuk 30 ms) dan satu interaksi nyata (mengganti jumlah pemain, termasuk menggambar ulang) selesai dalam **18 ms**. Server tetap melayani endpoint ringan seperti `/api/meta` tanpa masuk ke antrean mesin, karena endpoint itu memang tidak memanggil mesin. Yang berat tetap dibatasi: `numSims` ditangani berurutan (`DOMINO_MAX_ENGINE`, bawaan 1) dan permintaan yang menunggu lebih dari 60 detik dijawab 503, bukan menggantung selamanya.

Terakhir, jalur yang benar-benar dipakai pengguna: memilih tier 1.700.000 memunculkan peringatan (dan peringatan itu muncul kembali setiap kali pengguna menaikkan tier dari yang lebih ringan), analisis berjalan dengan indikator waktu, tombol Batal mematikan proses dalam ~0,4 detik, dan

hasilnya tampil: **7 langkah, interval ±0,03 poin, selesai 15,4 detik** dari perkiraan yang ditampilkan (~19 detik) — perkiraan yang sengaja dibuat sedikit pesimistis.

7.6 Uji antarmuka tanpa kepala (6 tes)

Peramban sungguhan tapi tanpa jendela (Playwright) menjalankan aplikasi yang utuh — server Python, `web/`, dan mesin C++ — supaya perilaku yang hidup di JavaScript tidak lagi hanya diuji manual:

```
pip install playwright
python tools/test_ui_heavy.py
```

Paket `playwright` memakai peramban yang sudah terpasang di komputer (Edge/Chrome), jadi tidak perlu mengunduh Chromium; bila tidak ada peramban sama sekali, uji ini **dilewati** (skip), bukan gagal. Di CI, Chromium dipasang eksplisit pada pekerjaan `ui-headless`.

Tes	Yang dibuktikan
test_heavy_option_exists_and_warns_before_use	opsi 1.700.000 ada; memilihnya memunculkan dialog berisi jumlah simulasi, perkiraan waktu, dan cara membatalkan; banner serta label sekali-jalan menyala; mode Live padam
test_declining_warning_reverts_to_previous_tier	menolak dialog mengembalikan tier sebelumnya tanpa mengubah mode Live; menyetujui dialog tetap membuka tier berat
test_button_label_always_matches_tier	label tombol selalu cocok dengan tier pada kenaikan/penurunan berturut-turut, dan peringatan muncul lagi di setiap kenaikan
test_manual_pause_survives_tier_change	jeda Live yang dilakukan pengguna sendiri tidak dinyalakan kembali saat tier diturunkan
test_heavy_tier_still_warned_after_reload	sesi tersimpan pada tier berat tetap menampilkan banner setelah halaman dimuat ulang, tanpa dialog yang menghadang
test_cancel_button_stops_engine_process	menekan Batal: proses domino_engine benar-benar hilang (~0,4 detik pada mesin uji), server mencatat [engine] dibatalkan, label pulih ke mode sekali-jalan, dan tidak ada hasil yang tampil

Tes terakhir berjalan dari posisi dengan 5 kandidat langkah (≈10–20 detik pada 1,7 juta simulasi) dan memeriksa bahwa analisis **masih berjalan** tepat sebelum Batal ditekan. Tanpa pemeriksaan itu, uji hanya akan membuktikan analisis yang kebetulan sudah selesai. Dengan uji ini, jalur soket yang sesungguhnya (klien menutup koneksi → server mematikan mesin) ikut terjaga, bukan hanya fungsi internalnya lewat uji unit.

7.6.1 Uji ini bergigi — dan itu diperiksa

Uji yang selalu hijau tidak membuktikan apa pun, jadi tiap klaim utama diuji dengan **mutasi**: satu aturan dicabut dari kode, dan uji harus gagal dengan gejala yang tepat.

Mutasi (cacat yang disuntikkan)	Gejala yang tertangkap

label tidak lagi mengikuti tier	label harus menandai analisis sekali-jalan, dapat: ' Analisis Live'
penolakan dialog tidak mengembalikan tier	'1700000' != '50000'
tier berat tidak lagi mematikan Live	label harus menandai analisis sekali-jalan, dapat: ' Jeda Live'
server mengabaikan koneksi yang ditutup klien	proses mesin masih hidup > 8 detik setelah Batal

Setelah kode dipulihkan, keenam uji kembali lulus.

7.7 Integrasi berkelanjutan (CI)

Setiap kali perubahan dikirim ke cabang `main`, empat pekerjaan berjalan di GitHub Actions:

Pekerjaan	Sistem	Yang dijalankan
engine-linux	Ubuntu, g++	bangun mesin, selftest, uji server
engine-windows	Windows, MSVC	bangun mesin, selftest, uji server
server-linux	Ubuntu, tanpa mesin	uji server saja (tes yang butuh mesin dilewati dengan rapi)
ui-headless	Ubuntu + Chromium	uji antarmuka tanpa kepala untuk tier 1,7 juta

Empat pekerjaan ini menjawab empat pertanyaan berbeda:

1. **Apakah mesin masih bisa dibangun oleh dua kompiler berbeda?** Ini menangkap penggunaan fitur bahasa yang tidak portabel.
1. **Apakah invarian mesin masih berlaku?** Ini yang menjalankan sebelas pemeriksaan.
2. **Apakah server tetap benar tanpa mesin?** Ini memastikan validasi, normalisasi, dan kontrak status HTTP tidak bergantung pada biner yang ada.
1. **Apakah alur tier berat tetap benar dari sisi pengguna?** Ini menjalankan peramban sungguhan yang menekan tombol-tombolnya: peringatan, penolakan, label, dan Batal yang mematikan proses mesin.

Perhatikan pekerjaan ketiga: uji server dirancang agar tes yang membutuhkan mesin **dilewati secara sadar** (`skipTest`) bila biner belum dibangun, bukan gagal. Prinsipnya sama seperti jalur cadangan pada pembagian kartu: kekurangan prasyarat dilaporkan, bukan disamarkan.

7.8 Bagaimana membuktikan hasil analisis bisa dipercaya

Ada prosedur tiga langkah yang bisa Anda jalankan sendiri, tanpa membaca kode:

Langkah 1 — buktikan determinisme. Jalankan permintaan yang sama dua kali dan bandingkan keluarannya:

```
python - <<'PY'
import json, subprocess
req = json.dumps({"cmd":"analyze","numPlayers":4,"cardsPerPlayer":7,"numSims":300,
                  "seed":123,"leftEnd":-1,"rightEnd":-1,
                  "myHand":["0-0","1-2","3-4"],"played":[],
                  "opponents":[{},{},{}]})
run = lambda: subprocess.run(["./bin/domino_engine.exe"],
                             input=req.encode(), capture_output=True).stdout
print("identik:", run() == run())
PY
```

Bila keluaran `identik: True`, mesin Anda bersikap deterministik.

Langkah 2 — periksa konservasi kartu. Setiap langkah pada balasan analisis memuat bidang `conserved`. Nilai `true` berarti tidak ada kartu yang muncul atau hilang selama simulasi untuk kandidat itu.

Langkah 3 — periksa kewajaran interval keyakinan. Untuk 500 simulasi dan peluang sekitar 70%, selang seharusnya berkisar sekitar 4 sampai 5 poin ke kiri dan ke kanan (misalnya 65-73%). Bila Anda melihat selang yang menempel ke 0% atau 100%, ada yang salah pada perhitungannya.

7.9 Sebuah cacat nyata: rumus interval keyakinan

Karena dokumentasi ini menyertakan contoh keluaran yang benar-benar dihitung mesin, penulisnya menemukan sesuatu saat menyusun Bagian V: **setiap interval keyakinan tampil sebagai 0,0% sampai 100,0%.** Peluang menangnya benar, tetapi rentang keyakinannya tidak berguna sama sekali.

Penyebabnya rumus Wilson yang ditulis salah: suku `p(1-p)` tidak dibagi jumlah simulasi, dan pembagi akhirnya dipasang di tempat yang salah, sehingga marginnya terlalu besar dan selalu terpotong ke batas 0-1. Perhatikan mengapa cacat ini bisa lolos: uji yang ada hanya memeriksa bahwa interval **mengurung** peluang terukur — dan `[0,1]` memang mengurung semuanya. Uji itu benar secara logika, tetapi terlalu longgar untuk menangkap kesalahannya.

Perbaikannya tiga hal sekaligus:

1. rumus Wilson dikoreksi di mesin (dan diberi komentar agar tidak terulang);
2. pemeriksaan mandiri nomor 11 ditambahkan ke mesin, yang memeriksa nilai interval pada kasus uji yang diketahui **dan** memastikan interval menyempit saat simulasi ditambah;
1. uji end-to-end server diperketat: untuk 200 simulasi, lebar interval kini harus di bawah 30 poin, sehingga interval yang menempel ke 0-100% akan gagal.

Pelajaran yang bisa dibawa ke pekerjaan apa pun: **uji yang hanya memeriksa batas lemah memberi rasa aman yang keliru.** Uji yang baik memeriksa sifat yang bermakna — di sini, bahwa intervalnya benar-benar menyempit ketika data bertambah.

7.10 Yang belum tercakup oleh pengujian otomatis

Kejujuran tentang cakupan sama pentingnya dengan hasil tesnya sendiri.

- **Tata letak antarmuka tidak punya uji otomatis.** Susunan ular, pengecilan papan, pemusatan kartu pertama, dan posisi siluet rekomendasi diverifikasi secara manual di peramban (termasuk pada lebar ponsel 390 piksel). Perubahan CSS atau JavaScript pada bagian itu tidak akan tertangkap oleh tes.
- **Sebagian interaksi pengguna kini dijaga otomatis, sebagian belum.** Alur tier

1,7 juta (peringatan, penolakan, label, Batal yang mematikan mesin) dijaga uji peramban tanpa kepala; modal, masukan cepat, dan alur Undo masih diuji manual.

- **Belum ada tolok ukur kinerja di CI.** Waktu tunggu analisis bergantung pada perangkat; angka waktu di Bagian IV adalah perkiraan, bukan hasil pengukuran otomatis.
- **Akurasi heuristik tidak diverifikasi terhadap permainan manusia.** Mesin memverifikasi konsistensi internal dan konservasi, bukan "apakah langkah yang disarankan benar-benar menang di meja nyata".
- **Pemeriksaan mandiri memakai pembangkit acak bersemai**, jadi hasilnya dapat diulang, tetapi cakupan skenarionya tetap terbatas pada skenario yang ditulis di kode.

Bila Anda ingin menambah pengujian, sasaran yang paling berharga saat ini adalah uji otomatis untuk tata letak papan dan uji interaksi dasar untuk alur kartu → PASS → atribusi → Undo (fondasi peramban tanpa kepala untuk itu sudah ada di `tools/test_ui_heavy.py`).

BAGIAN VIII

Panduan Strategi

VIII

Bagian ini adalah sisi manusiawi dari aplikasi: bagaimana memakai informasi yang dihitung mesin untuk benar-benar memenangkan permainan. Semua saran di sini sejalan dengan apa yang dihitung mesin, tetapi disampaikan sebagai prinsip yang bisa Anda bawa ke meja tanpa membuka layar.

8.1 Empat prinsip dasar

Prinsip 1: kuasai satu angka, bukan tujuh kartu. Kemenangan domino jarang datang dari kartu besar. Ia datang dari **angka** yang Anda kuasai. Bila Anda memegang empat kartu berangka 5, maka setiap kali ujung meja 5, Anda punya jalan keluar dan lawan sangat mungkin tidak. Karena itu: hitung angka, bukan nilai mata — kecuali permainan mendekati adu.

Prinsip 2: ujung adalah alat, bukan sekadar tempat menaruh kartu. Dua angka di ujung meja adalah satu-satunya kendali yang Anda punya atas langkah lawan. Setiap kali Anda memilih ujung, Anda sedang memilih **soal** yang harus dijawab lawan.

Prinsip 3: informasi adalah mata uang. Satu PASS lawan bernilai lebih dari satu kartu bagus. Karena itu, disiplin mencatat PASS dan atribusi kartu memberi keuntungan lebih besar daripada menambah jumlah simulasi.

Prinsip 4: jangan mengadu kecuali Anda ringan. Bila permainan macet, pemenangnya ditentukan nilai mata. Tangan berat harus berusaha membuka permainan; tangan ringan justru diuntungkan oleh kemacetan.

8.2 Mendominasi ujung

Angka yang Anda pegang tiga kali atau lebih disebut **dominan** (aplikasi menandainya di banner kualitas dan di panel dominasi). Dengan angka dominan, tujuan Anda adalah membuat angka itu muncul di ujung meja dan **mempertahankannya** di sana.

Mengapa? Karena lawan yang menyambung ke angka dominan Anda justru membantu Anda: rantai kembali ke angka Anda, dan Anda punya kartu untuk melanjutkannya. Sebaliknya, lawan yang tidak bisa menyambung harus PASS — dan itu bukti tambahan bahwa angka itu milik Anda.

Situasi terbaik yang bisa Anda capai: **kedua ujung meja adalah angka dominan Anda**. Mesin menghitung ini secara khusus dan memberi bonus besar (`domSupportScore` bernilai 30 untuk kondisi "kedua ujung"), karena lawan kini harus menjawab dengan angka yang Anda kendalikan dari dua arah. Bila panel hasil menampilkan lencana dominasi dengan kalimat "KEDUA UJUNG dikuasai angka X", itu lencana yang layak diperjuangkan.

8.3 Jerat: membuat lawan membuka jalan Anda

Jerat adalah langkah yang menyisakan angka **dominan Anda** di ujung sementara masih ada kartu berangka itu di luar (di tangan lawan atau sisa). Logikanya: siapa pun yang menyambung ke angka itu terpaksa mengeluarkan kartu berangka tersebut, sehingga persediaan lawan berkurang dan Anda semakin menguasai angka itu.

Mesin menandai jerat dengan `trapScore` dan alasan seperti "JERAT angka 6 terpasang! 4 kartu 6 masih di luar." Jerat bernilai tinggi bila kartu di luar banyak, karena setiap kartu yang keluar memperkecil peluang lawan bertahan.

Perhatikan bedanya dengan kesalahan yang mirip: menyisakan angka di ujung yang **bukan** milik Anda. Mesin memakai `selfTrapScore` untuk memperingatkan kondisi itu — contoh nyata di Bagian V menunjukkan kartu dengan win rate lebih tinggi justru kalah peringkat karena menyisakan ujung yang berbahaya bagi diri sendiri.

8.4 Kunci: menutup jalan lawan

Bila peta angka menunjukkan sebuah angka sudah **tidak ada lagi** di luar (tidak di tangan Anda dan tidak di papan), maka angka itu mati: tidak ada yang bisa menyambungnya. Membuat angka mati itu menjadi ujung berarti meja macet di sisi tersebut.

Kondisi paling kuat muncul ketika angka itu habis dari peredaran **tetapi Anda masih memegang kartunya**. Mesin menyebut ini "KUNCI" (`blockScore` 25) karena hanya Anda yang punya akses. Bagi lawan, ujung itu adalah jalan buntu.

```
Peta angka:
  3:  di tangan saya 2, sudah keluar 5, sisa 0
      -> angka 3 habis dari peredaran
Meja : ujung kiri = 3
Anda : punya kartu 3-6
Artinya: Anda memegang satu-satunya akses ke ujung kiri.
```

Blok terjamin. Bila Anda sudah mencatat PASS lawan, mesin bisa menyatakan lebih tegas lagi: "Lawan 2 PASTI tidak bisa main (tidak punya 3 maupun 1)". Ini berlaku keras karena didasarkan pada bukti, bukan perkiraan. Langkah seperti ini biasanya menjadi alasan peringkat #1 meskipun win rate-nya bukan yang tertinggi, karena lawan yang mati berarti giliran kembali ke Anda.

8.5 Adu: bermain dengan sisa mata

Adu bukan kegagalan permainan; ia adalah akhir permainan yang berbeda, dengan aturan penilaian yang berbeda. Tiga hal yang perlu Anda ingat:

1. **Setel konvensi meja lebih dulu.** "Mata terkecil" dan "rata-rata" memberi hasil

berbeda, dan pilihan "seri = menang/kalah" juga berpengaruh. Aplikasi memakai pilihan Anda secara konsisten.

1. **Nilai mata terkecil menang.** Karena itu, bila permainan mengarah ke adu, kartu besar adalah beban. Membuang balak besar lebih awal adalah investasi.
1. **Paksa macet bila Anda ringan.** Bila nilai mata tangan Anda kecil, permainan macet menguntungkan Anda. Mesin menampilkan bar "Menang lewat ADU" untuk setiap langkah sehingga Anda bisa melihat langkah mana yang paling mendorong skenario itu.

Balak punya peran khusus di sini: balak bernilai dua kali angkanya (`6-6` = 12, yang terbesar di set), jadi banyak balak berarti adu yang buruk. Penilaian kualitas kartu di antarmuka secara eksplisit menandai tangan dengan empat balak atau lebih sebagai "KARTU JELEK (BANYAK BALAK)" dengan saran membuang balak cepat-cepat.

8.6 Cangkul: menyambar dengan sadar biaya

Cangkul mengubah perhitungan secara mendasar, karena kartu sisa bisa berpindah tangan. Dua hal yang sering dilupakan:

- **Kartu sisa memperpanjang permainan.** Setiap kali seseorang menyambar, jumlah kartu tersembunyi berkurang dan informasi Anda bisa berubah. Catat atribusi setiap sambaran yang Anda lihat; itu menghapus kartu dari kolam kemungkinan lawan.
- **Bila cangkul mati, kartu sisa justru aset strategis.** Kartu di sisa tidak akan pernah keluar. Bila sisa memuat banyak kartu berangka tertentu, angka itu menjadi langka di permainan nyata — dan itu berarti meja lebih mudah macet, sehingga peluang adu lebih tinggi. Panel "Kartu Sisa Mati" menampilkan perkiraan ini per angka.

8.7 Menyesuaikan diri dengan gaya lawan

Mesin mensimulasikan tiga gaya lawan supaya rekomendasinya tidak bergantung pada satu pola main. Anda bisa memakai ketiga gaya itu sebagai kacamata membaca manusia di meja.

Gaya	Ciri di meja	Yang harus Anda lakukan
Blocker (penahan)	sering menyisakan ujung yang sepi, suka membuat orang PASS	buka jalur alternatif; jangan bergantung pada satu ujung
Hoarder (penimbun)	menyimpan kartu dari angka kuatnya, lambat membuang	jangan menyerahkan ujung ke angka kuatnya; paksa dia membuang
Dumper (pembuang)	cepat membuang kartu besar dan balak	dia akan habis cepat, jadi percepat jalur Anda sendiri; siapkan adu bila ragu

Bila salah satu lawan bermain persis seperti **penahan**, aktifkan mode **teliti**. Mode itu membuat lawan yang bermain tepat setelah Anda membalas dengan permainan ketat, bukan dengan pilihan acak, sehingga estimasi Anda lebih konservatif dan lebih dekat kenyataan.

8.8 Rutinitas satu langkah

Inilah kebiasaan yang membuat pemakaian aplikasi benar-benar menguntungkan. Lakukan empat langkah ini setiap giliran; total waktunya di bawah sepuluh detik setelah terbiasa.

1. **Perbarui data dulu, jangan langsung lihat hasil.** Sebelum memutuskan, pastikan semua

yang sudah terjadi sudah tercatat: kartu lawan yang Anda lihat, dan PASS lawan. Analisis yang dihitung dari data lama adalah analisis yang salah.

1. **Baca tiga hal di panel hasil:** peringkat #1, rentang keyakinannya, dan alasan yang tertulis. Bila #1 dan #2 punya selang yang saling menumpuk, anggap keduanya sama baik.
1. **Periksa bar "Keamanan Putaran 2"** (muncul di mode kartu pertama). Langkah dengan win rate tinggi tetapi keamanan nol akan membuat Anda PASS di putaran kedua.
1. **Cocokkan dengan akal sehat.** Bila rekomendasi bertentangan dengan pengamatan meja Anda (misalnya leluhur lawan selalu menyimpan angka tertentu), percayai meja — lalu perbaiki data yang Anda masukkan, karena hampir selalu penyebabnya adalah data yang kurang lengkap.

8.9 Kesalahan umum dan cara menghindarinya

Kesalahan	Akibat	Perbaikan
Menandai PASS "untuk berjaga-jaga"	simulasi menghormati bukti palsu; rekomendasi menyimpang	tandai PASS hanya bila benar-benar terlihat
Lupa mencatat kartu lawan	profil lawan menyesatkan, kemungkinan kartu tidak menyempit	catat setiap kartu lawan yang bisa Anda atribusikan
Salah jumlah kartu per pemain	jumlah kartu tersembunyi dan sisa tidak sinkron	cek baris keterangan di panel Setup
Memakai 1000 simulasi lalu memercayai selisih 2%	derau statistik dianggap sinyal	naikkan ke 10.000, atau lihat interval keyakinan
Mengubah pengaturan meja di tengah permainan	hasil sebelum dan sesudah tidak bisa dibandingkan	setel aturan sebelum mencatat
Mengabaikan mode kartu pertama	kartu buka yang buruk membuat putaran kedua macet	baca bar keamanan putaran 2 sebelum membuka
Menimpa sesi berharga dengan Reset	data hilang	tekan Export dulu sebelum Reset

8.10 Membaca situasi dengan cepat

Tiga pertanyaan yang bisa Anda ajukan ke peta angka dalam hitungan detik:

1. **Apakah angka di ujung masih hidup?** Bila "sisa 0" dan tidak ada di tangan Anda, ujung itu mati untuk semua orang — permainan akan macet.
1. **Apakah saya memegang angka yang sudah langka?** Semakin kecil "sisa", semakin besar nilai kartu Anda di angka itu.
1. **Apakah lawan kehabisan kartu berangka tertentu?** Lihat panel lawan: angka yang tertulis "pasti tidak punya" tidak akan pernah muncul dari lawan itu. Bila angka itu adalah satu-satunya penghubung, arahkan permainan ke sana.

Dengan tiga pertanyaan itu saja, Anda sudah memakai kekuatan utama aplikasi ini tanpa harus membaca satu pun angka statistik. Sisanya adalah memperhalus keputusan, dan itu tugas Bagian V.

BAGIAN IX

Rujukan API

IX

Bagian ini ditujukan untuk siapa pun yang ingin memakai aplikasi ini dari skrip, alat monitoring, atau integrasi lain. Seluruh API berbasis JSON sederhana: kirim objek, terima objek.

9.1 Aturan dasar

Aspek	Ketentuan
Alamat dasar	http://127.0.0.1:2024 (bawaan)
Format isi	application/json — wajib; tipe isi lain ditolak dengan 400
Asal permintaan	bila header Origin dikirim, harus sama dengan Host; selain itu ditolak 400
Ukuran maksimum isi	1 MiB
Autentikasi	hanya bila APP_TOKEN diisi: header X-App-Token pada semua POST /api/*
Token pembanding	waktu-konstan (hmac.compare_digest)
Batas beban	numSims antara 1 dan 1700000; mesin dihentikan bila melewati 300 detik
Header balasan	X-Content-Type-Options: nosniff, Referrer-Policy: no-referrer, CSP ketat
Tempat singgahan	balasan API Cache-Control: no-store; berkas statis no-cache

9.2 Daftar endpoint

Metode	Jalur	Kegunaan
GET	/ atau /index.html	antarmuka web
GET	/<berkas>	berkas statis dari folder web/
GET	/api/health	status server dan kesiapan mesin
POST	/api/analyze	analisis Monte-Carlo untuk seluruh kandidat langkah

POST	/api/selftest	menjalankan sebelas pemeriksaan mandiri mesin
POST	/api/meta	daftar kartu yang belum terlihat (tidak di tangan, tidak di papan)
—	/api/... lain	404, {"ok": false, "error": "endpoint tidak dikenal"}
GET	/api/meta	405, karena endpoint ini hanya menerima POST

9.3 GET /api/health

Tidak butuh isi. Berguna untuk memeriksa kesiapan sebelum mengirim permintaan berat.

```
curl -s http://127.0.0.1:2024/api/health
```

```
{
  "ok": true,
  "server": "python",
  "version": 3,
  "engine": "domino_engine.exe",
  "engineReady": true,
  "tokenRequired": false
}
```

Bidang	Arti
engine	nama berkas mesin, atau null bila belum dibangun
engineReady	false berarti analisis akan dijawab 503
tokenRequired	true berarti APP_TOKEN aktif dan header token wajib

9.4 POST /api/meta

Menghitung kartu yang belum diketahui: 28 kartu dikurangi kartu di tangan Anda dan kartu yang sudah di papan. Kunci kartu pada masukan akan dinormalkan seperti biasa.

```
curl -s -X POST http://127.0.0.1:2024/api/meta \
  -H 'Content-Type: application/json' \
  -d '{"myHand":["6-1","0|0"],"played":[]}'
```

```
{"ok": true, "remaining": ["0-1","0-2","0-3","...","6-6"]}
```

Endpoint ini selalu mengembalikan 28 kartu bila tangan dan papan kosong. Balasannya tidak pernah memuat kartu mustahil (seperti `0-7`) — hal itu pernah menjadi cacat dan sudah diperbaiki serta dijaga oleh uji `meta_only_real_tiles`.

9.5 POST /api/analyze — permintaan

9.5.1 Bidang yang wajib

Bidang	Tipe	Batasan	Arti
numPlayers	angka	2, 3, atau 4	jumlah pemain di meja, termasuk Anda
cardsPerPlayer	angka	1 sampai 14, dan numPlayers x cardsPerPlayer tidak boleh melebihi 28	kartu yang dibagikan ke setiap pemain
myHand	larik kunci kartu	tidak boleh berisi duplikat, tidak boleh tumpang tindih dengan played	kartu di tangan Anda
played	larik kunci kartu	idem	kartu yang sudah masuk papan, urut kronologis

9.5.2 Bidang opsional

Bidang	Tipe	Bawaan	Arti
numSims	angka	10000	jumlah simulasi per kandidat langkah (1 sampai 1700000)
seed	angka	42	seed pembangkit acak; aplikasi web memakai 20240922
leftEnd / rightEnd	angka	−1	angka di ujung kiri dan kanan papan; −1 berarti papan kosong
playedBy	larik label	semua "unknown"	pemilik setiap kartu di played; panjangnya harus sama; nilai sah: "me", "unknown", "opp1" sampai "opp4"
nextSeat	angka	1	indeks pemain yang bermain setelah Anda (0 berarti Anda sendiri)
deadlockRule	teks	"lowest"	"lowest" (mata terkecil) atau "average" (rata-rata)
tieRule	teks	"win"	"win" atau "lose" untuk nilai mata yang seri
cangkul	boolean	true	apakah kartu sisa boleh disambar
adversarial	boolean	false	mode teliti: lawan yang bermain setelah Anda bermain tanpa derau
opponents	larik objek	kosong	satu objek per lawan, berurutan; lihat di bawah

9.5.3 Tier akurasi numSims (varians maksimum, p = 0,5)

Tingkat kepercayaan	Margin of error	Jumlah simulasi minimal	Tier di aplikasi
95%	±3,1%	~1.000	1.000 (cepat)
95%	±1,4%	~5.000	5.000
95%	±1,0%	9.604 (≈10 ribu)	10.000 (bawaan)
95%	±0,44%	~50.000	50.000
95%	±0,1%	960.400 (≈1 juta)	dicakup oleh tier 1.700.000
99%	±0,1%	1.664.100 (≈1,7 juta)	1.700.000 (dengan peringatan; lihat bab antarmuka)
99%	±0,01%	166.410.000 (≈166 juta)	tidak ditawarkan — berjam-jam per permintaan

Rumus: $N = Z^2 \times p(1-p) / E^2$, dengan Z = 1,96 (95%) atau 2,58 (99%). Pada mesin uji (6 inti fisik / 12 utas logis) mesin mengerjakan ~0,6–1,4 juta simulasi-langkah per detik (lihat `tools/bench_engine.py`), sehingga tier 1.700.000 memakan **7 detik** untuk 3 kandidat langkah dan **~30 detik** untuk 12 kandidat langkah; efisiensi paralelnya 87% dari 6 inti fisik. Karena itu tier ini tidak berjalan otomatis: antarmuka meminta konfirmasi, mematikan mode Live, dan menyediakan tombol Batal yang benar-benar menghentikan proses mesin.

9.5.4 Bentuk objek lawan

```
{"passes": 1, "eliminated": [3, 1]}
```

Bidang	Arti
passes	jumlah PASS yang tercatat untuk lawan ini (dipakai antarmuka untuk menampilkan riwayat)
eliminated	daftar angka 0-6 yang dipastikan tidak dipegang lawan ini, hasil pencatatan PASS

Angka pada `eliminated` menjadi batasan keras di dalam simulasi: kartu yang mengandung angka tersebut tidak akan pernah dibagikan ke lawan itu, kecuali kuota kartunya mustahil dipenuhi tanpa melanggar (dalam hal itu batasannya dilepas dan kejadiannya dihitung sebagai *fallback*).

9.5.5 Format kunci kartu

Ditulis	Diterima menjadi
3-5, `3	5, 3:5, 3.5, 3 5, 35`
6-1, `6	1, 61`
7-1, a-b, 1-2-3, kosong, angka bukan teks	ditolak: 400 dengan pesan yang menyebut posisi elemen

> Bila Anda memanggil **mesin** secara langsung (bukan lewat server), normalisasi ini tidak terjadi: mesin hanya menerima bentuk kanonik. `"6-3"` akan ditolak dengan `invalid tile key: 6-3`.

9.6 POST /api/analyze — balasan

Balasan berisi dua bagian: **ringkasan permintaan** (supaya Anda bisa memastikan mesin memahami apa yang Anda maksud) dan **daftar langkah**.

```
{
  "ok": true,
  "engine": "domino-cpp/3.0.0",
  "numPlayers": 4, "cardsPerPlayer": 7, "numSims": 500, "seed": 20240922,
  "leftEnd": 3, "rightEnd": 1, "nextSeat": 1,
  "adversarial": false, "deadlockRule": "lowest", "tieRule": "win", "cangkul": true,
  "isFirstMove": false, "hasPassData": true, "hasAttribution": true,
  "unknownCount": 20, "totalOppCards": 20, "boneyardCount": 0, "validMoveCount": 3,
  "opponents": [ ... ],
  "moves": [ ... ],
  "deadTiles": [ ... ],
  "deadByNumber": [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
}
```

Bidang ringkasan	Arti
engine	versi mesin yang menjawab
isFirstMove	true bila papan masih kosong (mode kartu pertama)
hasPassData	true bila ada data PASS yang dipakai
hasAttribution	true bila ada kartu yang diatribusikan ke lawan
unknownCount	jumlah kartu yang belum diketahui (belum di tangan, belum di papan)
totalOppCards	perkiraan jumlah kartu yang ada di tangan semua lawan
boneyardCount	jumlah kartu sisa/cangkul menurut pembagian
validMoveCount	jumlah kandidat langkah yang sah

9.6.1 Bidang setiap langkah (moves[])

Bidang	Arti
key, a, b	kartu kandidat
side	"left", "right", atau "first"
newLeft, newRight	ujung papan setelah langkah ini dimainkan
wins, losses, numSims	jumlah menang, kalah, dan total simulasi
winRate	peluang menang (0 sampai 1)
winLo, winHi	batas bawah dan atas interval keyakinan Wilson 95%
rankScore	nilai peringkat gabungan (lihat Bagian V)
avgMyVal	rata-rata nilai mata di tangan Anda saat gim berakhir
avgOppMinVal	rata-rata nilai mata lawan terendah saat gim berakhir

flexibility	berapa kartu sisa yang masih bisa disambungkan setelah langkah ini
blockedGames, blockRate	berapa sering gim berakhir macet
deadlockWins, deadlockWinRate	kemenangan yang diperoleh lewat adu
winByEmpty, winByValue, winByEmptyRate	menang karena kartu habis vs menang karena nilai mata
riskScore	seberapa sering kartu berbahaya muncul pada gim yang kalah
topDanger	lima kartu berbahaya: [{"key": "3-5", "freq": 0.96}]
avgOppPasses	rata-rata berapa kali lawan PASS per gim
avgDraws	rata-rata jumlah sambaran cangkul per gim
avgDomControl	rata-rata berapa kali angka dominan Anda berada di ujung
conserved	true bila konservasi kartu terjaga selama simulasi langkah ini
domSupportScore, domSupportReasons	penilaian dan alasan dukungan dominasi
trapScore, trapReasons	penilaian dan alasan jerat
blockScore, blockReasons	penilaian dan alasan kunci
selfTrapScore	penilaian jebakan untuk diri sendiri
guaranteedBlocks	blok terjamin: [{"opp": 2, "reason": "Lawan 2 PASTI tidak bisa main (...)"}]
firstMoveSafety	hanya pada mode kartu pertama; berisi canPlayNext, totalRemaining, safetyRatio, canPlayKeys, cannotPlayKeys

Daftar `moves[]` sudah terurut menurun menurut `rankScore`, dan sifat ini dijaga oleh pemeriksaan mandiri nomor 7.

9.6.2 Bidang setiap lawan (opponents[])

Bidang	Arti
handSize	perkiraan jumlah kartu tersembunyi di tangan lawan itu
poolSize	jumlah kartu yang masih mungkin ia pegang setelah semua batasan
passElimCount	berapa angka yang tereliminasi dari PASS lawan ini
heldKnown	berapa kartu yang sudah dipastikan miliknya lewat atribusi Anda
doublesExpected	perkiraan jumlah balak di tangannya
dominant	dua angka teratas: [{"num": 5, "expected": 6.0, "pct": 100.0}]

9.6.3 Kartu sisa mati

`deadTiles` dan `deadByNumber` **hanya ada** bila masih ada kartu tersembunyi. Bila semua kartu sudah diketahui, kedua bidang itu tidak dikirim.

Bidang	Arti

deadTiles	daftar kartu beserta peluang berakhir di sisa: [{"key":"5-6","a":5,"b":6,"prob":0.42}], terurut menurun
deadByNumber	larik tujuh angka: perkiraan jumlah kartu mati untuk angka 0 sampai 6

9.7 POST /api/selftest

Menjalankan pemeriksaan mandiri mesin. Isi boleh objek kosong.

```
curl -s -X POST http://127.0.0.1:2024/api/selftest \
  -H 'Content-Type: application/json' -d '{}'
```

```
{
  "ok": true,
  "engine": "domino-cpp/3.0.0",
  "allPass": true,
  "checks": [
    {"name": "deck_28_kartu", "pass": true, "detail": "28 tile unik 0-0 .. 6-6"},
    {"name": "interval_wilson", "pass": true, "detail": "w=346/500 -> 0.650..0.731, lebar menyempit
     saat n naik"}
  ]
}
```

Bila `allPass` bernilai `false`, status HTTP yang dikirim adalah **500** — kegagalan pemeriksaan mandiri adalah masalah mesin, bukan kesalahan permintaan.

9.8 Contoh lengkap dengan curl

Contoh ini mereproduksi persis contoh di Bagian V.

```
curl -s -X POST http://127.0.0.1:2024/api/analyze \
  -H 'Content-Type: application/json' \
  -d '{
    "numPlayers": 4, "cardsPerPlayer": 7, "numSims": 500, "seed": 20240922,
    "leftEnd": 3, "rightEnd": 1,
    "myHand": ["6-6", "3-6", "2-3", "1-5", "4-4"],
    "played": ["0-3", "0-2", "1-2"],
    "playedBy": ["opp1", "opp2", "opp1"],
    "nextSeat": 1, "deadlockRule": "lowest", "tieRule": "win", "cangkul": true,
    "opponents": [{"passes": 1, "eliminated": [3, 1]}, {"passes": 0, "eliminated": []},
                  {"passes": 0, "eliminated": []}]
  }' | python -c "import json,sys; d=json.load(sys.stdin);
print([[m['key'], round(m['winRate']*100,1), round(m['winLo']*100,1), round(m['winHi']*100,1), m['r
ankScore']] for m in d['moves']])"
```

Keluaran yang diharapkan (persis seperti contoh di Bagian V):

```
[['3-6', 69.2, 65.0, 73.1, 84.155], ['2-3', 71.2, 67.1, 75.0, 60.444], ['1-5', 60.0, 55.6, 64.2, 51
.285]]
```

Contoh dengan token:

```
APP_TOKEN=rahasia python server/domino_server.py     # jendela server
curl -s -X POST http://127.0.0.1:2024/api/analyze \
  -H 'Content-Type: application/json' -H 'X-App-Token: rahasia' \
  -d '{"numPlayers":2,"cardsPerPlayer":7,"myHand":["0-0"],"played":[]}'
```

9.9 Kode kesalahan

Setiap kesalahan dikirim sebagai objek JSON dengan `ok: false` dan pesan `error` dalam bahasa Indonesia yang bisa dibaca manusia.

Status	Pesan contoh	Penyebab
400	body JSON tidak valid	tipe isi bukan JSON, isi kosong, terlalu besar, atau JSON rusak
400	myHand/played: format kartu tidak dikenal (elemen ke-2)	kunci kartu tidak sah
400	ada kartu duplikat di myHand	kartu yang sama dua kali
400	kartu tidak bisa di tangan dan di papan sekaligus	tumpang tindih antara myHand dan played
400	numSims harus angka 1..1700000	batas beban dilanggar
400	nextSeat harus angka 0..3	indeks pemain di luar batas
400	playedBy hanya boleh 'me', 'unknown', atau 'opp1'..'opp4'	label pemain tidak dikenal
400	tieRule harus 'win' atau 'lose'	nilai aturan tidak dikenal (ditolak mesin)
401	token tidak valid atau hilang (header X-App-Token)	APP_TOKEN aktif dan token salah
404	endpoint tidak dikenal	jalur /api/... tidak ada
405	gunakan POST untuk /api/meta	metode salah
500	engine error (exit 3) — detail ada di log server	mesin gagal berjalan; detail hanya di log
502	engine menghasilkan output tidak valid	mesin berjalan tetapi keluarannya bukan JSON
503	engine belum di-build. Jalankan: python tools/build_engine.py	biner mesin tidak ada
504	engine timeout (>300s)	permintaan melebihi 300 detik

Prinsip yang dijaga oleh pengujian: **`ok: false` selalu berpasangan dengan status bukan 200.** Klien boleh memutuskan berdasarkan kode status saja.

9.10 Memanggil mesin langsung

Mesin bisa dipakai tanpa server sama sekali — berguna untuk pengujian dan skrip batch.

```
echo '{"cmd":"analyze","numPlayers":2,"cardsPerPlayer":7,"numSims":200,"seed":1,
       "leftEnd":-1,"rightEnd":-1,"myHand":["0-0","1-2"],"played":[],
       "opponents":[{}]}' | ./bin/domino_engine          # Linux/macOS
```

```
'{"cmd":"selftest"}' | .\bin\domino_engine.exe            # Windows PowerShell
```

Aspek	Perilaku
Perintah	"cmd": "analyze" (bawaan) atau "cmd": "selftest"

Masukan	satu objek JSON dari masukan standar
Keluaran	satu objek JSON ke keluaran standar, diakhiri baris baru
Kode keluar	selalu 0; kesalahan dilaporkan sebagai ok:false di dalam JSON
Normalisasi kartu	tidak ada — kunci harus sudah kanonik (3-6, bukan 6-3)
Validasi	tetap ketat; numSims dibatasi 1 sampai 1700000 di dalam mesin
Batas waktu	tidak ada; pemanggil yang menentukan (server memakai 300 detik)

Karena kode keluarnya selalu 0, **jangan** menyimpulkan keberhasilan dari kode keluar saja. Periksa bidang `ok` di dalam balasan. Server sudah melakukan hal itu: ia memetakan `ok:false` dari mesin menjadi status HTTP 400, bukan 200.

BAGIAN X

Pemecahan Masalah & Lampiran

10.1 Pemecahan masalah

Masalah dikelompokkan menurut gejala yang Anda lihat. Kolom terakhir adalah perbaikan yang paling sering benar.

10.1.1 Saat menjalankan

Gejala	Kemungkinan penyebab	Perbaikan
TIDAK ADA compiler yang bisa dipakai (MSVC/g++/clang++)	tidak ada compiler C++ terpasang	pasang Visual Studio Build Tools (komponen C++) atau MinGW/g++
[start] build engine gagal.	kesalahan kompilasi atau compiler tidak lengkap	jalankan python tools/build_engine.py --force untuk melihat pesan kesalahan penuh
[selftest] ... GAGAL	mesin terpasang tetapi tidak konsisten (jarang)	bangun ulang dengan --force; bila tetap gagal, laporkan keluaran selftest
[start] Port 2024 sudah dipakai — server lain mungkin berjalan.	sudah ada server domino berjalan	tutup jendela server lama, atau jalankan server manual dengan DOMINO_PORT lain
Server hidup tetapi halaman kosong	berkas belum termuat atau tembolok lama	muat ulang dengan Ctrl+F5, atau naikkan nama tembolok di web/sw.js
Address already in use saat menjalankan server manual	port sedang dipakai proses lain	pilih port lain: DOMINO_PORT=8080 python server/domino_server.py

10.1.2 Saat memakai

Gejala	Kemungkinan penyebab	Perbaikan
"Tidak bisa menghubungi server"	server mati atau port berbeda	jalankan python start.py lagi
Analisis menjawab 503	biner mesin belum dibangun	python tools/build_engine.py --force

Analisis menjawab 401	APP_TOKEN aktif, antarmuka tidak mengirim token	kosongkan APP_TOKEN atau lindungi di lapisan tunnel
Bilah kemajuan berhenti di 30%	analisis sangat berat atau komputer sibuk	tunggu, turunkan jumlah simulasi, atau tekan tombol untuk membatalkan
Perubahan web/ tidak muncul	service worker menyajikan versi tembolok	ubah nama tembolok di web/sw.js, lalu muat ulang paksa
Papan meluber dan perlu digulir	isi papan melebihi batas pengecilan minimum	ini perilaku cadangan yang disengaja; papan tetap bisa digulir agar tidak ada kartu hilang
Siluet rekomendasi menghilang tiba-tiba	kartu rekomendasi sudah tidak bisa dimainkan lagi pada keadaan papan sekarang	jalankan analisis ulang (mode hidup biasanya sudah melakukannya)
Papan terlihat goyang saat jendela diubah	penataan sedang berlangsung	tunggu sekitar 150 milidetik; penataan ulang ditunda agar tidak berkedip
Angka peluang berbeda padahal data sama	jumlah simulasi berbeda, atau pengaturan meja berubah	samakan numSims dan pengaturan; hasilnya deterministik pada konfigurasi sama

10.2 Pertanyaan yang sering diajukan

Apakah aplikasi ini mengintip kartu lawan? Tidak. Aplikasi hanya memakai informasi yang Anda masukkan. Ia menghitung peluang, bukan mengetahui kartu.

Mengapa tombol analisis sudah menyala saat aplikasi dibuka? Karena mode hidup aktif sejak awal. Teks pada tombol itu adalah perintah untuk menjeda, bukan untuk memulai.

Apakah saya harus menekan tombol analisis setiap kali? Tidak. Setiap perubahan informasi memicu analisis ulang otomatis. Tombolnya hanya untuk menjeda.

Kenapa rekomendasinya jadi #2 padahal peluang menangnya lebih tinggi? Itulah gunanya rankScore: langkah yang peluangnya nyaris sama bisa dibedakan oleh jerat, kunci, blok terjamin, dan risiko. Bagian V memuat contoh nyata dengan angka lengkap.

Berapa jumlah simulasi yang sebaiknya dipakai? 2000 untuk pemakaian sehari-hari; 5000 bila Anda ingin memisahkan selisih tipis dan tidak keberatan menunggu.

Bisakah saya bermain 2 pemain? Bisa. Pilih 2 pemain; jumlah kartu maksimum menjadi 14 per pemain dan sisanya menjadi cangkul (kecuali Anda mematikannya).

Apa arti "tereliminasi 12%" di panel lawan? Berapa persen kemungkinan kartu lawan itu sudah berhasil dipersempit oleh data PASS dan atribusi yang Anda catat.

Apakah data saya dikirim ke internet? Tidak. Server hanya berjalan di komputer Anda. Berkas sesi disimpan di penyimpanan lokal peramban.

Bisa dipakai di ponsel tanpa memasang apa pun? Bisa, asalkan ponsel dapat menjangkau server di komputer Anda (misalnya lewat alamat IP jaringan lokal). Untuk akses dari luar jaringan, lihat bagian tunnel di Bagian III.

Bagaimana kalau saya salah mencatat PASS? Tekan **Undo** satu kali; PASS terakhir dibatalkan bersama eliminasi angkanya. Perhatikan tombol Undo di kartu lawan membatalkan PASS lawan itu, sedangkan tombol Undo utama membatalkan aksi terakhir secara umum.

Mengapa ada kartu yang ditandai "BALAK"? Karena balak bernilai dua kali angkanya dan sering menjadi beban bila permainan berakhir dengan adu. Penanda itu membantu Anda mendahulukan pembuangannya.

Apakah hasilnya bisa saya simpan untuk dianalisis ulang? Bisa. Tekan **Export** untuk mengunduh berkas JSON sesi, dan **Import** untuk memuatnya kembali di lain waktu.

Apa arti "konservasi" yang muncul di keluaran mesin? Penanda bahwa jumlah kartu tetap 28 selama simulasi: tidak ada kartu yang muncul atau hilang. Bila bernilai salah, hasil simulasi itu tidak layak dipercaya.

10.3 Glosarium

Istilah	Arti
Adu	akhir permainan karena meja macet; pemenang ditentukan nilai mata
Atribusi	menandai kartu di papan sebagai milik lawan tertentu
Balak	kartu dengan angka kembar, misalnya 5-5
Boneyard	kartu sisa/cangkul yang belum dibagikan
Cangkul	menyambar kartu sisa setelah tidak bisa jalan
Constraint-aware dealing	pembagian kartu dalam simulasi yang menghormati bukti PASS dan atribusi
Deterministik	masukan sama menghasilkan keluaran sama, berkat seed tetap
Fallback	kejadian ketika batasan simulasi harus dilepas karena kuota kartu mustahil dipenuhi
Ghost / siluet	gambar garis putus-putus di papan yang menunjukkan tempat rekomendasi
Interval keyakinan (Wilson)	rentang tempat nilai sebenarnya kemungkinan besar berada
Invarian	sifat yang harus selalu benar; diperiksa oleh selftest
Jerat (trap)	menyisakan angka dominan di ujung agar lawan terpaksa membukanya
Klik	sebutan lain untuk kartu domino
Kunci (block)	ujung yang hanya bisa diakses Anda
Mata	nilai satu sisi kartu
Mode teliti (adversarial)	simulasi dengan lawan yang bermain ketat tanpa derau
Monte-Carlo	metode menaksir peluang dengan menjalankan banyak percobaan acak
Nilai mata	jumlah kedua angka sebuah kartu atau tangan
PASS	tidak bisa jalan; menjadi bukti bahwa angka ujung tidak dipegang
Peta angka	panel yang menunjukkan sebaran kartu per angka 0-6
rankScore	nilai peringkat gabungan yang dihitung mesin

Siku	kartu berdiri di tikungan susunan ular
Sisa mati	kartu sisa yang tidak pernah masuk permainan
Ujung	angka bebas di kiri atau kanan rantai meja
Ular (boustrophedon)	susunan baris bergantian arah seperti ular
Win rate	peluang menang hasil simulasi untuk sebuah langkah

10.4 Lampiran A — seluruh 28 kartu

Kartu	Nilai mata	Kartu	Nilai mata	Kartu	Nilai mata	Kartu	Nilai mata
0-0	0	1-2	3	2-4	6	4-5	9
0-1	1	1-3	4	2-5	7	4-6	10
0-2	2	1-4	5	2-6	8	5-5	10
0-3	3	1-5	6	3-3	6	5-6	11
0-4	4	1-6	7	3-4	7	6-6	12
0-5	5	2-2	4	3-5	8	—	—
0-6	6	2-3	5	3-6	9	—	—
1-1	2	—	—	4-4	8	—	—

Balak: `0-0` (0), `1-1` (2), `2-2` (4), `3-3` (6), `4-4` (8), `5-5` (10), `6-6` (12). Kartu bernilai tertinggi adalah `6-6` (12); kartu bernilai 0 hanyalah `0-0`.

10.5 Lampiran B — pembagian kartu dan sisa

Pemain	Kartu per pemain	Dibagikan	Sisa/cangkul	Kartu tersembunyi per lawan (perkiraan)
2	7	14	14	7
2	10	20	8	10
3	7	21	7	7
3	9	27	1	9
4	5	20	8	5
4	7	28	0	7

"Kartu tersembunyi per lawan" adalah jumlah kartu yang tidak Anda lihat di tangan satu lawan; angka inilah yang dipakai mesin untuk menghitung kuota pembagian.

10.6 Lampiran C — rumus dan bobot penting

rankScore (dihitung mesin, dipakai antarmuka untuk mengurutkan):

```
rankScore = winRate x 100 + domSupportScore x 0,6 + trapScore x 0,8 + blockScore x 0,7
          - selfTrapScore x 0,5 - riskScore x 0,5 + deadlockWinRate x 20
          + |guaranteedBlocks| x 8 + avgOppPasses x 3 + safetyRatio x 30 (kartu pertama)
```

Interval keyakinan Wilson 95% (rumus yang dipakai setelah perbaikan):

```
pusat  = (p + z²/2n) / (1 + z²/n)
margin = z x akar( p(1-p)/n + z²/4n² ) / (1 + z²/n)      z = 1,96
bawah  = max(0, pusat - margin)      atas = min(1, pusat + margin)
```

Skor risiko: jumlah lima frekuensi kartu teratas yang dimainkan lawan pada gim yang Anda kalah, masing-masing dikali 10.

Keamanan kartu pertama: jumlah kartu sisa di tangan yang masih bisa dimainkan setelah kandidat pertama, dibagi jumlah kartu sisa.

Konservasi: `kartu di semua tangan + kartu di papan = kartu saat pembagian`.

10.7 Lampiran D — catatan versi dan perbaikan

Tanggal/periode	Perubahan
v1	analisis domino dasar dalam satu program
v2	pemisahan menjadi mesin C++ + server Python; simulasi, peta angka, konvensi meja
v3	atribusi kartu per lawan, tiga profil lawan, mode teliti, Wilson, rankScore, PWA
v3.1	mode cangkul dan kartu sisa mati, kisi kandidat kartu per lawan, masukan angka
v3.2	mode hidup aktif sejak pembukaan, masukan cepat, indikator status di judul
v3.3	siluet rekomendasi, glow biru kartu pilihan, perbaikan pilihan ujung untuk kartu lawan
v3.6	papan bergaya ular, pengecilan otomatis, pemusatan kartu pertama
audit keamanan	perbaikan XSS pada pesan galat, proteksi CSRF/DoS, penanganan badan rusak, kode status HTTP yang benar
regresi	perbaikan pemberian kemenangan akhir permainan yang salah
perbaikan rumus interval	rumus Wilson dikoreksi: setiap interval keyakinan sebelumnya selalu 0-100% sehingga tidak berguna; kini benar dan menyempit sesuai jumlah simulasi, dengan pemeriksaan mandiri baru untuk menjaganya

Perbaikan terakhir itu ditemukan justru saat dokumentasi ini disusun, karena contoh keluaran nyata diperiksa satu per satu sebelum ditulis ke buku. Kisah lengkapnya ada di Bagian VII.

10.8 Lampiran E — membangun ulang dokumen ini

Buku yang sedang Anda baca dihasilkan oleh dua skrip, bukan ditulis dua kali:

```
pip install fpdf2 python-docx        # dependensi build dokumen
python tools/build_docs.py           # Markdown gabungan + PDF + DOCX
python tools/build_deck.py           # presentasi PPTX
```

Berkas	Peran
docs/src/*.md	sumber isi buku (sepuluh bagian)
docs/Domino-Analyzer-Pro-Dokumentasi.md	buku gabungan dalam Markdown
docs/Domino-Analyzer-Pro-Dokumentasi.pdf	buku cetak dengan daftar isi bernomor halaman dan diagram vektor
docs/Domino-Analyzer-Pro-Dokumentasi.docx	versi yang bisa diedit
docs/Domino-Analyzer-Pro-Presentasi.pptx	presentasi ringkas

Pembangun PDF memeriksa lebih dulu apakah font yang tersedia bisa menampilkan seluruh karakter yang dipakai; karakter yang tidak didukung akan diganti padanannya, dan bila masih ada yang tidak bisa ditampilkan, hal itu dilaporkan alih-alih dibiarkan menjadi kotak kosong di dokumen. Perintah pemeriksaan saja:

```
python tools/build_docs.py --check-fonts
```

10.9 Lisensi dan kontribusi

Proyek ini **belum menyertakan berkas lisensi**. Secara hukum, tanpa lisensi eksplisit hak cipta tetap berada pada penulisnya dan penggunaan ulang tidak otomatis diizinkan. Bila Anda ingin membagikan atau memakai ulang proyek ini secara luas, langkah yang disarankan adalah menambahkan berkas `LICENSE` (misalnya MIT untuk pemakaian bebas, atau GPL bila Anda ingin turunannya tetap terbuka).

Bagi yang ingin berkontribusi, urutan yang paling berguna untuk diperiksa sebelum mengirim perubahan:

1. `python tools/selftest.py` — sebelas pemeriksaan mesin harus lulus;
2. `python tools/test_server.py` — 29 uji server harus lulus;
3. `python tools/test_engine_parallel.py` — 8 uji paralel/determinisme harus lulus (tambahkan `DOMINO_SLOW_TESTS=1` untuk ikut menguji beban penuh 1,7 juta simulasi);
1. `python tools/test_ui_heavy.py` — 6 uji antarmuka tanpa kepala untuk tier 1,7 juta harus lulus (perlu `pip install playwright`; dilewati bila tidak ada peramban);
1. `python tools/build_engine.py --force` — pastikan mesin masih bisa dibangun dari nol.

Perubahan pada tata letak papan sebaiknya disertai pemeriksaan manual pada lebar ponsel dan desktop, karena tata letak belum punya uji otomatis. Perubahan pada alur tier 1,7 juta (peringatan, label tombol, pembatalan) sudah punya pengaman otomatis — jalankan `tools/test_ui_heavy.py`.

10.10 Tentang penulis

Dody Maulana Ramadhan

Aplikasi **Domino Analyzer Pro** beserta dokumentasi ini dibuat oleh Dody Maulana Ramadhan: mesin analisis Monte-Carlo dalam C++20, server API Python, antarmuka web PWA, serta seluruh perkakas pengujian, integrasi berkelanjutan, dan buku panduan ini.

Buku ini disusun sebagai penjelasan lengkap sekaligus panduan pemakaian, dengan satu prinsip yang dipegang dari halaman pertama sampai terakhir: **setiap angka yang ditulis di sini harus bisa dilacak kembali ke kode yang menghasilkannya.** Contoh keluaran di Bagian V dan Bagian IX diambil dari eksekusi mesin yang sesungguhnya, bukan dari ilustrasi.

10.11 Penutup

Domino adalah permainan informasi yang tidak lengkap. Kita tidak pernah tahu tujuh kartu lawan, dan justru di situlah letak keseruannya. Yang bisa dilakukan pemain yang disiplin adalah memastikan **informasi yang tersedia tidak terbuang**: setiap PASS dicatat, setiap kartu lawan diatribusikan, dan setiap keputusan diambil dengan kesadaran penuh atas peluang dan ketidakpastiannya.

Alat ini dibuat untuk tujuan itu — bukan untuk menggantikan kepekaan seorang pemain, melainkan untuk menutup celah yang paling sulit ditutup oleh manusia: menghitung. Selamat bermain, dan semoga papan Anda selalu memberi jalan.